МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
им. М.В. Ломоносова

АНО «Инженерная академия»

А. А. Часовских, В. С. Половников

ПРАКТИКУМ

по нейронным сетям

Учебно-методическое пособие для студентов
направления «Прикладная математика и информатика»

Москва, 2026

УДК 004.85

ББК 32.813

АННОТАЦИЯ

Данное учебно-методическое пособие предназначено для студентов старших курсов и аспирантов, изучающих курс «Нейронные сети» в рамках направления «Прикладная математика и информатика». Пособие содержит 10 лабораторных работ, охватывающих ключевые темы современной глубокой обработки изображений: от базовых свёрточных сетей до современных генеративных моделей.

По завершении курса студенты будут способны разрабатывать и обучать нейронные сети для задач классификации и детекции, применять Transfer Learning для работы с малыми данными, реализовывать генеративные модели (VAE, GAN, Diffusion), анализировать и интерпретировать результаты обучения.

АВТОРЫ

Часовских Анатолий Александрович
доктор физико-математических наук, профессор,
кафедра математической теории интеллектуальных систем (МаТИС)
механико-математического факультета МГУ имени М.В. Ломоносова

Половников Владимир Сергеевич
кандидат физико-математических наук, старший научный сотрудник,
кафедра математической теории интеллектуальных систем (МаТИС)
механико-математического факультета МГУ имени М.В. Ломоносова

Учебно-методическое пособие. 50 академических часов.
Google Colab + PyTorch

Оглавление

Введение **7**

Общие положения **9**
Назначение пособия . . . 9
Цели практикума . . . 9
Как пользоваться пособием . . . 9

Организация занятий **11**
Формат проведения . . . 11
Структура занятия (5 часов) . . . 11
Технические требования . . . 11

Контроль и оценивание **13**
Лабораторные работы . . . 13
Критерии оценки лабораторных работ . . . 13
Мини-проект (отчётное задание) . . . 14
Общие требования . . . 14
Темы мини-проектов (на выбор) . . . 14
Критерии оценки мини-проекта . . . 14

Термины и определения **15**

Сокращения **16**

Литература и ресурсы **17**
Основная литература . . . 17
Дополнительная литература . . . 17
Онлайн-ресурсы . . . 17
Репозиторий курса . . . 18

Занятие 1. Введение в CNN. Процесс обучения в PyTorch **19**
Цель занятия . . . 19
Теоретическая часть (1 час) . . . 19
Архитектура свёрточных сетей . . . 19
Пайплайн данных в PyTorch . . . 19
Цикл обучения . . . 20
Практическая часть (3 часа) . . . 20
Шаг 1. Настройка окружения . . . 20
Шаг 2. Загрузка данных . . . 21
Шаг 3. Определение модели CNN . . . 22
Шаг 4. Настройка обучения . . . 23

Шаг 5. Функции обучения и валидации . . . 23
Шаг 6. Цикл обучения . . . 24
Шаг 7. Визуализация результатов . . . 26
Шаг 8. Визуализация предсказаний . . . 26
Шаг 9. Визуализация карт признаков . . . 27
Домашнее задание . . . 28
Контрольные вопросы . . . 28
Критерии сдачи занятия . . . 29

Занятие 2. Переобучение и регуляризация **30**
Цель занятия . . . 30
Теоретическая часть (1 час) . . . 30
Проблема переобучения . . . 30
Методы регуляризации . . . 30
Практическая часть (3 часа) . . . 31
Шаг 1. Подготовка урезанного датасета . . . 31
Шаг 2. Базовая модель (без регуляризации) . . . 31
Шаг 3. Эксперимент 1: Демонстрация переобучения . . . 32
Шаг 4. Эксперимент 2: Dropout и Weight Decay . . . 33
Шаг 5. Эксперимент 3: Аугментация данных . . . 34
Шаг 6. Сравнение результатов . . . 34
Домашнее задание . . . 36
Контрольные вопросы . . . 36

Занятие 3. Transfer Learning: Feature Extraction vs Fine-tuning **37**
Цель занятия . . . 37
Теоретическая часть (1 час) . . . 37
Идея Transfer Learning . . . 37
Архитектура ResNet . . . 38
Практическая часть (3 часа) . . . 38
Шаг 1. Загрузка датасета . . . 38
Шаг 2. Загрузка предобученной модели . . . 39
Шаг 3. Стратегия 1: Feature Extraction (Freeze) . . . 40
Шаг 4. Обучение (Feature Extraction) . . . 40
Шаг 5. Стратегия 2: Fine-tuning . . . 41
Шаг 6. Сравнение стратегий . . . 42
Домашнее задание . . . 43
Контрольные вопросы . . . 43
Критерии сдачи занятия . . . 43

Занятие 4. Оптимизация, Дистилляция и Mutual Learning **44**
Цель занятия . . . 44
Теоретическая часть (1 час) . . . 44
Оптимизаторы и Scheduler . . . 44
Дистилляция знаний (Knowledge Distillation) . . . 44
Взаимное обучение (Deep Mutual Learning) . . . 45
Практическая часть (3 часа) . . . 45
Шаг 1. Подготовка данных и моделей . . . 45
Шаг 2. Обучение Teacher модели . . . 46
Шаг 3. Реализация Distillation Loss . . . 47
Шаг 4. Реализация Mutual Learning . . . 48

Шаг 5. Сравнение результатов . . . 50
Домашнее задание . . . 51
Контрольные вопросы . . . 51

Занятие 5. Детектирование объектов (YOLO) 52
Цель занятия . . . 52
Теоретическая часть (1 час) . . . 52
Принцип работы YOLO . . . 52
Метрики качества . . . 52
Практическая часть (3 часа) . . . 53
Шаг 1. Установка библиотеки . . . 53
Шаг 2. Загрузка предобученной модели . . . 53
Шаг 3. Подготовка датасета . . . 53
Шаг 4. Fine-tuning модели . . . 54
Шаг 5. Оценка и визуализация . . . 55
Шаг 6. Анализ результатов . . . 55
Домашнее задание . . . 56
Контрольные вопросы . . . 56
Критерии сдачи занятия . . . 56

Занятие 6. Трансформеры в компьютерном зрении (ViT) 57
Цель занятия . . . 57
Теоретическая часть (1 час) . . . 57
Архитектура ViT . . . 57
Механизм Attention . . . 57
Практическая часть (3 часа) . . . 58
Шаг 1. Подготовка данных (CIFAR-10) . . . 58
Шаг 2. Реализация Patch Embedding . . . 58
Шаг 3. Сборка модели ViT . . . 59
Шаг 4. Обучение модели . . . 60
Шаг 5. Визуализация Attention Maps . . . 61
Домашнее задание . . . 62
Контрольные вопросы . . . 62
Критерии сдачи занятия . . . 63

Занятие 7. Автоэнкодеры (AE) и латентные пространства 64
Цель занятия . . . 64
Теоретическая часть (1 час) . . . 64
Принцип работы AE . . . 64
Применение . . . 64
Практическая часть (3 часа) . . . 65
Шаг 1. Загрузка данных (MNIST) . . . 65
Шаг 2. Реализация простого AE . . . 65
Шаг 3. Обучение AE . . . 66
Шаг 4. Визуализация реконструкции . . . 67
Шаг 5. Визуализация латентного пространства (PCA) . . . 67
Шаг 6. Denoising Autoencoder (DAE) . . . 68
Домашнее задание . . . 69
Контрольные вопросы . . . 69
Критерии сдачи занятия . . . 70

Занятие 8. Вариационные автоэнкодеры (VAE) 71
Цель занятия . . . 71
Теоретическая часть (1 час) . . . 71
Принцип работы VAE . . . 71
Функция потерь VAE . . . 71
Преимущества VAE . . . 71
Практическая часть (3 часа) . . . 72
Шаг 1. Загрузка данных (FashionMNIST) . . . 72
Шаг 2. Реализация VAE . . . 72
Шаг 3. Функция потерь VAE . . . 73
Шаг 4. Обучение модели . . . 74
Шаг 5. Генерация новых изображений . . . 74
Шаг 6. Интерполяция в латентном пространстве . . . 75
Шаг 7. Визуализация функции потерь . . . 76
Домашнее задание . . . 76
Контрольные вопросы . . . 76
Критерии сдачи занятия . . . 76

Занятие 9. Генеративно-состязательные сети (GAN) 78
Цель занятия . . . 78
Теоретическая часть (1 час) . . . 78
Принцип работы GAN . . . 78
Проблемы обучения GAN . . . 78
DCGAN Architecture . . . 78
Практическая часть (3 часа) . . . 79
Шаг 1. Загрузка данных (FashionMNIST) . . . 79
Шаг 2. Реализация Генератора . . . 79
Шаг 3. Реализация Дискриминатора . . . 80
Шаг 4. Инициализация весов . . . 80
Шаг 5. Функции потерь и оптимизаторы . . . 81
Шаг 6. Цикл обучения GAN . . . 81
Шаг 7. Визуализация прогресса генерации . . . 82
Шаг 8. Графики функции потерь . . . 83
Шаг 9. Сравнение VAE и GAN . . . 83
Домашнее задание . . . 83
Контрольные вопросы . . . 83
Критерии сдачи занятия . . . 84

Занятие 10. Диффузионные модели (DDPM / LDM) 85
Цель занятия . . . 85
Теоретическая часть (1 час) . . . 85
Принцип диффузии . . . 85
Latent Diffusion Models (LDM) . . . 85
Практическая часть (3 часа) . . . 86
Шаг 1. Подготовка данных и гиперпараметров . . . 86
Шаг 2. Функции добавления шума . . . 86
Шаг 3. Модель UNet (Упрощённая) . . . 87
Шаг 4. Цикл обучения . . . 88
Шаг 5. Процесс сэмплирования (Генерация) . . . 89
Шаг 6. Сравнение генеративных моделей . . . 90

Домашнее задание . . . 90
Контрольные вопросы . . . 91
Критерии сдачи занятия . . . 91

Мини-проект: Методические указания **92**
Общие сведения . . . 92
Этапы выполнения . . . 92
Требования к коду . . . 92
Требования к отчёту . . . 92

Материалы курса в GitHub **94**
Репозиторий курса . . . 94
Структура репозитория . . . 94
Как работать с репозиторием . . . 94
Клонирование . . . 94
Получение обновлений . . . 94
Отправка решений (форк) . . . 95

Приложение А. Шаблон отчёта по лабораторной работе **96**

Приложение Б. Чек-лист перед сдачей **97**

Приложение В. Примеры тем мини-проектов **98**

Заключение **99**

Введение

Машинное обучение и искусственные нейронные сети переживают период взрывного роста. За последние годы архитектуры нейронных сетей совершили качественный скачок: свёрточные сети достигли сверхчеловеческой точности в задачах распознавания изображений, трансформеры стали универсальным инструментом не только для обработки последовательностей, но и изображений и видео, а генеративные модели научились создавать фотореалистичные изображения, видео и даже трёхмерные объекты. Практическое применение этих технологий охватывает всё новые сферы деятельности — от медицинской диагностики и разработки лекарств до автономного транспорта и творческих индустрий. При этом темпы развития области таковы, что методы, считавшиеся передовыми год назад, сегодня могут быть вытеснены новыми архитектурами, что делает непрерывное образование необходимым условием профессиональной востребованности специалиста.

Данное учебно-методическое пособие содержит 10 лабораторных работ, охватывающих ключевые архитектуры и методы современного глубокого обучения: свёрточные нейронные сети для классификации изображений, методы регуляризации и борьбы с переобучением, Transfer Learning для эффективной работы с ограниченными данными, дистилляцию знаний и совместное обучение для создания компактных моделей, детектирование объектов архитектурами семейства YOLO, Vision Transformer для обработки изображений, автоэнкодеры и вариационные автоэнкодеры для обучения представлений, генеративно-состязательные сети и диффузионные модели для генерации данных. Пособие призвано научить студентов практическому применению теоретических знаний: от построения архитектуры нейронной сети до обучения, отладки и интерпретации результатов, формируя компетенции, необходимые для самостоятельной исследовательской и инженерной работы в области искусственного интеллекта.

Хотя данное пособие сфокусировано на обработке изображений, изучаемые методы и архитектуры имеют универсальный характер и успешно обобщаются на другие предметные области. Свёрточные сети применяются к одномерным данным при анализе временных рядов и аудиосигналов, к трёхмерным данным в медицинской томографии и компьютерной графике, к графовым структурам в химии и биоинформатике. Автоэнкодеры позволяют находить компактные представления сложных объектов для сжатия данных и обнаружения аномалий. Трансформеры, изначально созданные для обработки текста, стали универсальной архитектурой для мультимодальных систем. Ярким примером междисциплинарного переноса технологий стала Нобелевская премия по химии 2024 года, присуждённая за применение методов машинного обучения, включая диффузионные модели, к задаче предсказания структуры белков — это демонстрирует, как подходы, разработанные для генерации изображений, находят применение в фундаментальной науке.

Вместе с тем невозможно получить полноценное владение технологиями машинного обучения без практической работы с нейронными сетями. На практике специалисты сталкиваются с множеством технических нюансов, которые невозможно освоить из теоретических курсов: выбор архитектуры и гиперпараметров для конкретной зада-

чи, правильная предобработка и аугментация данных, диагностика проблем обучения (vanishing gradients, mode collapse, overfitting), интерпретация метрик и визуализация результатов. Эти навыки приобретаются в основном в процессе написания кода, проведения экспериментов, анализа ошибок и их исправления. При этом практическая подготовка важна не только для инженеров, но и для исследователей-теоретиков: без понимания технических ограничений и особенностей реализации невозможно разрабатывать новые методы, которые будут применимы на практике. Теоретическая и практическая подготовки неразрывно связаны — первая даёт понимание принципов, вторая формирует умение применять эти принципы для решения реальных задач.

Данное учебно-методическое пособие может быть рекомендовано к использованию в системе дополнительного профессионального образования, а также для самостоятельного изучения студентами старших курсов и аспирантами, желающими углубить свои компетенции в области нейронных сетей. Каждая лабораторная работа построена по принципу «от теории к практике»: сначала излагаются основные концепции и математические основы, затем следует детальное руководство по реализации с комментариями к каждой строке кода. Практическая часть выполняется в среде Google Colab с использованием GPU, что делает пособие доступным без необходимости установки специализированного программного обеспечения. По завершении курса студенты будут способны самостоятельно разрабатывать и обучать нейронные сети, применять Transfer Learning для работы с малыми данными, реализовывать генеративные модели и анализировать результаты обучения.

Материалы курса (шаблоны ноутбуков, дополнительные примеры, обновления) публикуются в открытом репозитории GitHub:

https://github.com/V171/neural-networks-practicum

Общие положения

Назначение пособия

Настоящее пособие регламентирует организацию и проведение практикума по нейронным сетям. Документ содержит:

- цели и задачи практикума;
- порядок проведения занятий;
- требования к выполнению лабораторных работ;
- критерии оценки результатов обучения;
- методические указания к каждому занятию.

Цели практикума

1. Формирование практических навыков реализации нейронных сетей в PyTorch.
2. Изучение современных архитектур: CNN, ViT, YOLO, VAE, GAN, Diffusion.
3. Освоение процессов обучения, регуляризации и отладки моделей.
4. Развитие компетенций в области компьютерного зрения и генеративных моделей.
5. Подготовка к самостоятельной исследовательской работе (мини-проект).

Как пользоваться пособием

Перед началом курса:

- Создайте аккаунт Google (если отсутствует).
- Зарегистрируйтесь на Google Colab: https://colab.research.google.com
- Создайте папку на Google Drive для хранения работ.
- Клонируйте репозиторий курса:
  `git clone https://github.com/V171/neural-networks-practicum`

Перед каждым занятием:

- Изучите теоретический материал по теме занятия.
- Откройте шаблон ноутбука из репозитория курса.

- Подготовьте вопросы по непонятным моментам.

Во время занятия:

- Выполняйте код пошагово, фиксируя результаты.
- Делайте заметки в ячейках Markdown.
- Сохраняйте промежуточные веса моделей.

После занятия:

- Завершите обучение модели (если требуется).
- Оформите отчёт в том же ноутбуке.
- Загрузите работу в систему до дедлайна.

Организация занятий

Формат проведения

Параметр	Очное занятие	Удалённое (ВКС)
Продолжительность	5 академических часов	5 академических часов
Платформа	Аудитория с ПК	Zoom/Teams + Colab
Связь	Локальная сеть	Видеоконференция
Контроль	Личный	Демонстрация экрана
Консультации	В аудитории	Чат ВКС / Email

Таблица 1: Форматы проведения занятий

Структура занятия (5 часов)

1. **Теоретическая часть (1 час):**
   - Лекция по теме занятия.
   - Разбор ключевых концепций.
   - Ответы на вопросы.
2. **Практическая часть (3 часа):**
   - Работа с Colab-ноутбуком.
   - Реализация моделей в PyTorch.
   - Обучение и визуализация результатов.
3. **Подведение итогов (1 час):**
   - Проверка результатов.
   - Обсуждение проблем.
   - Инструкция по домашнему заданию.

Технические требования

- **Браузер:** Google Chrome (последняя версия).
- **Аккаунт:** Google аккаунт для доступа к Colab.
- **Runtime:** GPU T4 (Runtime → Change runtime type → GPU).

- **Хранилище:** Google Drive (минимум 5 ГБ свободно).
- **Интернет:** Стабильное соединение для загрузки датасетов.

Контроль и оценивание

Лабораторные работы

№	Тема	Часы
1	Введение в CNN	5
2	Переобучение и регуляризация	5
3	Transfer Learning	5
4	Дистилляция и Mutual Learning	5
5	Детектирование (YOLO)	5
6	Трансформеры (ViT)	5
7	Автоэнкодеры (AE)	5
8	Вариационные автоэнкодеры (VAE)	5
9	Генеративно-состязательные сети (GAN)	5
10	Диффузионные модели	5
Итого		**50**

Таблица 2: Перечень лабораторных работ

Критерии оценки лабораторных работ

Зачёт:

- Код выполняется без ошибок.
- Присутствуют все требуемые визуализации.
- Есть текстовые выводы по результатам.
- Модель достигает целевых метрик.

Незачёт:

- Код не запускается.
- Отсутствуют обязательные графики.
- Нет анализа результатов.
- Метрики ниже пороговых значений.

Допуск к мини-проекту: выполнение минимум 8 из 10 лабораторных работ.

Мини-проект (отчётное задание)

Общие требования

- **Срок выполнения:** после завершения 50 часов практикума.
- **Формат:** Colab-ноутбук + презентация (5–7 слайдов).
- **Объём:** эквивалент 20–30 часов самостоятельной работы.
- **Защита:** устная презентация + демонстрация модели.

Темы мини-проектов (на выбор)

1. Сравнение AE, VAE, GAN, Diffusion на одном датасете.
2. Реализация Conditional Diffusion Model (генерация по классу).
3. Детектирование специфических объектов (свой датасет).
4. Исследование Mutual Learning на разных архитектурах.
5. Адаптация YOLO для детекции аномалий.
6. Transfer Learning для медицинской диагностики.
7. Интерполяция в латентном пространстве VAE.
8. Стилизация изображений с помощью нейросетей.

Критерии оценки мини-проекта

Критерий	**Вес**
Актуальность и сложность задачи	20%
Корректность реализации	30%
Качество результатов (метрики)	25%
Оформление отчёта и презентации	15%
Защита (ответы на вопросы)	10%
Итого	**100%**

Таблица 3: Критерии оценки мини-проекта

Термины и определения

Нейронная сеть Вычислительная система, вдохновлённая биологическими нейронными сетями, состоящая из взаимосвязанных узлов (нейронов).

Свёрточный слой (Conv2d) Слой нейронной сети, применяющий операцию свёртки для извлечения пространственных признаков из изображений.

Функция потерь (Loss Function) Мера ошибки модели, которую необходимо минимизировать в процессе обучения.

Обратное распространение (Backpropagation) Алгоритм вычисления градиентов функции потерь по параметрам сети.

Переобучение (Overfitting) Ситуация, когда модель хорошо работает на обучающих данных, но плохо обобщается на новые данные.

Регуляризация Методы предотвращения переобучения (Dropout, BatchNorm, L2).

Transfer Learning Использование предобученной модели для решения новой задачи.

Дистилляция знаний Процесс обучения компактной модели (student) с помощью большой модели (teacher).

Mutual Learning Совместное обучение нескольких моделей, использующих предсказания друг друга.

Автоэнкодер (AE) Нейронная сеть для обучения эффективных представлений данных без учителя.

VAE Вариационный автоэнкодер — вероятностная версия автоэнкодера.

GAN Генеративно-состязательная сеть — система из генератора и дискриминатора.

Диффузионная модель Генеративная модель, основанная на процессе постепенного добавления и удаления шума.

Attention Механизм взвешивания важности различных частей входных данных.

ViT Vision Transformer — архитектура трансформера для обработки изображений.

YOLO You Only Look Once — семейство одностадийных детекторов объектов.

mAP mean Average Precision — метрика качества детектирования объектов.

IoU Intersection over Union — мера перекрытия двух bounding box.

Сокращения

Сокращение	**Расшифровка**
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
ViT	Vision Transformer
AE	AutoEncoder
VAE	Variational AutoEncoder
GAN	Generative Adversarial Network
DDPM	Denoising Diffusion Probabilistic Model
LDM	Latent Diffusion Model
YOLO	You Only Look Once
mAP	mean Average Precision
IoU	Intersection over Union
LR	Learning Rate
BN	Batch Normalization
KL	Kullback-Leibler divergence
MSE	Mean Squared Error
CE	Cross Entropy
GPU	Graphics Processing Unit
Colab	Google Colaboratory

Таблица 4: Перечень сокращений

Литература и ресурсы

Основная литература

1. Гудфеллоу Я., Бенджио И., Курвилль А. «Глубокое обучение». — ДМК Пресс, 2017.
2. Шолле Ф. «Глубокое обучение на Python». — Питер, 2018.
3. Хайкин С. «Нейронные сети: полный курс». — Вильямс, 2006.

Дополнительная литература

1. Половников В. С. «Об оптимизации структурной реализации нейронных сетей». — Диссертация, 2007.
2. Lecun Y. et al. «Gradient-Based Learning Applied to Document Recognition». — IEEE, 1998.
3. He K. et al. «Deep Residual Learning for Image Recognition». — arXiv:1512.03385.
4. Vaswani A. et al. «Attention Is All You Need». — arXiv:1706.03762.
5. Dosovitskiy A. et al. «An Image is Worth 16x16 Words». — arXiv:2010.11929.
6. Ho J. et al. «Denoising Diffusion Probabilistic Models». — arXiv:2006.11239.

Онлайн-ресурсы

- Официальная документация PyTorch: https://pytorch.org/docs/
- PyTorch Tutorials: https://pytorch.org/tutorials/
- Google Colab: https://colab.research.google.com/
- Kaggle: https://www.kaggle.com/
- Papers with Code: https://paperswithcode.com/
- Hugging Face: https://huggingface.co/
- Ultralytics YOLO: https://docs.ultralytics.com/

Репозиторий курса

Материалы курса (шаблоны ноутбуков, примеры решений, обновления) доступны в открытом репозитории:

`https://github.com/V171/neural-networks-practicum`

Структура репозитория:

- `notebooks/` — шаблоны ноутбуков для каждого занятия
- `data/` — ссылки на датасеты и примеры данных
- `solutions/` — примеры решений (доступны после дедлайнов)
- `utils/` — вспомогательные функции
- `docs/` — дополнительная документация

Занятие 1. Введение в CNN. Процесс обучения в PyTorch

Цель занятия

- Освоить базовый пайплайн работы с данными в PyTorch.
- Реализовать свёрточную нейронную сеть с нуля.
- Написать полный цикл обучения модели.
- Получить первую обученную модель с accuracy $> 85\%$.

Теоретическая часть (1 час)

Архитектура свёрточных сетей

Свёрточная нейронная сеть (CNN) состоит из следующих типов слоёв:

1. **Conv2d** — свёрточный слой для извлечения признаков. Применяет набор фильтров к входному изображению, каждый фильтр выделяет определённый паттерн (края, текстуры, формы).
2. **ReLU** — функция активации $f(x) = \max(0, x)$. Вводит нелинейность в сеть, позволяя обучать сложные зависимости.
3. **MaxPool2d** — слой подвыборки для уменьшения размерности карты признаков. Выбирает максимальное значение в каждом окне, что делает представление инвариантным к небольшим сдвигам.
4. **Linear** — полносвязный слой для классификации. Преобразует вектор признаков в логиты классов.

Пайплайн данных в PyTorch

```python
import torch
from torch.utils.data import DataLoader, Dataset
from torchvision import datasets, transforms

# Определение трансформаций для предобработки данных
# ToTensor() преобразует изображение в тензор [0, 1]
# Normalize() нормализует данные: (x - mean) / std
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # Нормализация к [-1, 1]
])

# Загрузка датасета FashionMNIST
```

```
# Датасет содержит 70000 ч/б изображений 28x28 (10 классов)
train_dataset = datasets.FashionMNIST(
    root='./data',      # Папка для сохранения данных
    train=True,         # Обучающая выборка (60000 изображений)
    download=True,      # Скачать из интернета, если нет локально
    transform=transform # Применить трансформации
)

# Создание DataLoader для пакетной загрузки
# DataLoader разбивает данные на батчи и перемешивает
train_loader = DataLoader(
    train_dataset,
    batch_size=64,      # Размер батча
    shuffle=True,       # Перемешивать данные
    num_workers=0       # Без многопоточности
)
```

Ожидаемый результат: После выполнения кода будет скачан датасет FashionMNIST (около 30 МБ) и создан объект DataLoader, который позволяет итерироваться по батчам данных. При каждой эпохе данные будут перемешиваться.

Цикл обучения

Основные этапы одного шага обучения:

1. **Forward pass** — вычисление предсказаний модели.
2. **Вычисление функции потерь (loss)** — мера расхождения между предсказаниями и истинными метками.
3. **Backward pass** — вычисление градиентов функции потерь по всем параметрам модели.
4. **Обновление весов** — корректировка параметров в направлении антиградиента.

Практическая часть (3 часа)

Шаг 1. Настройка окружения

```
# Проверка доступности GPU
# GPU ускоряет обучение нейронных сетей в 10-100 раз
import torch

print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"Device: {torch.cuda.get_device_name(0)}")
else:
    print("Device: CPU")

# Фиксация random seed для воспроизводимости результатов
# Важно для научных экспериментов и отладки
import random
import numpy as np

def set_seed(seed=42):
    """Установка seed для всех источников случайности"""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
        # Для полной детерминированности (может замедлить)
        torch.backends.cudnn.deterministic = True

set_seed(42)
print("Seed установлен: 42")
```

Ожидаемый результат:

- Вывод информации о доступном GPU (например, Tesla T4) или CPU
- Установка seed для воспроизводимости — при повторных запусках с тем же seed результаты будут идентичны

Шаг 2. Загрузка данных

```python
from torchvision import datasets, transforms

# Трансформации для тренировочных данных
# Для обучения добавляем аугментацию (расширение данных)
train_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # Нормализация к диапазону [-1, 1]
])

# Трансформации для тестовых данных
# Для теста аугментация не нужна - только нормализация
test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Загрузка FashionMNIST
# Классы: 0-T-shirt, 1-Trouser, 2-Pullover, 3-Dress, 4-Coat,
#         5-Sandal, 6-Shirt, 7-Sneaker, 8-Bag, 9-Ankle boot
train_dataset = datasets.FashionMNIST(
    root='./data',
    train=True,
    download=True,
    transform=train_transform
)

test_dataset = datasets.FashionMNIST(
    root='./data',
    train=False,
    download=True,
    transform=test_transform
)

# Создание DataLoader
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=0
)

print(f"Train samples: {len(train_dataset)}")   # Ожидается: 60000
print(f"Test samples: {len(test_dataset)}")     # Ожидается: 10000
print(f"Количество батчей: {len(train_loader)}")  # Ожидается: 938
```

Ожидаемый результат:

- Загрузка датасета FashionMNIST (60000 train + 10000 test изображений)
- Размер батча 64 → 938 батчей для обучения
- Каждое изображение — тензор размерности (1, 28, 28)

Шаг 3. Определение модели CNN

```python
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    """
    Простая свёрточная нейронная сеть для классификации FashionMNIST.

    Архитектура:
    - 3 свёрточных блока (Conv -> ReLU -> MaxPool)
    - 2 полносвязных слоя с Dropout

    Вход: (batch, 1, 28, 28)
    Выход: (batch, 10) - логиты для 10 классов
    """
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()

        # Свёрточные слои для извлечения признаков
        # Conv2d(in_channels, out_channels, kernel_size, padding)
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)   # 28x28 -> 28x28
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)  # 14x14 -> 14x14
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1) # 7x7 -> 7x7

        # Слой подвыборки (уменьшает размер в 2 раза)
        self.pool = nn.MaxPool2d(2, 2)

        # Полносвязные слои для классификации
        # После 3 пулингов: 28 -> 14 -> 7 -> 3
        self.fc1 = nn.Linear(128 * 3 * 3, 256)
        self.fc2 = nn.Linear(256, num_classes)

        # Dropout для регуляризации (предотвращение переобучения)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        """
        Прямой проход сети.

        Размерности тензоров:
        Вход: (B, 1, 28, 28)
        После conv1+pool: (B, 32, 14, 14)
        После conv2+pool: (B, 64, 7, 7)
        После conv3+pool: (B, 128, 3, 3)
        После flatten: (B, 1152)
        После fc1: (B, 256)
        После fc2: (B, 10)
        """
        # Conv блок 1: извлечение низкоуровневых признаков (края, углы)
        x = self.pool(F.relu(self.conv1(x)))  # 28x28 -> 14x14

        # Conv блок 2: извлечение среднеуровневых признаков (формы)
        x = self.pool(F.relu(self.conv2(x)))  # 14x14 -> 7x7

        # Conv блок 3: извлечение высокоуровневых признаков (объекты)
        x = self.pool(F.relu(self.conv3(x)))  # 7x7 -> 3x3

        # Преобразование в плоский вектор
        x = x.view(-1, 128 * 3 * 3)

        # Полносвязные слои
        x = self.dropout(F.relu(self.fc1(x)))  # С Dropout для регуляризации
        x = self.fc2(x)

        return x

# Создание модели
model = SimpleCNN(num_classes=10)
print(model)

# Подсчёт количества параметров модели
def count_parameters(model):
    """Подсчёт обучаемых параметров модели"""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

total_params = count_parameters(model)
print(f"Total trainable parameters: {total_params:,}")
```

```
# Ожидается: ~460,000 параметров
```

Ожидаемый результат:

- Создана модель SimpleCNN с 3 свёрточными и 2 полносвязными слоями
- Общее количество параметров: около 460,000
- Вывод структуры модели в консоль

Шаг 4. Настройка обучения

```
# Выбор устройства для вычислений
# GPU (cuda) ускоряет обучение в 10-100 раз
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
print(f"Модель перемещена на: {device}")

# Функция потерь для многоклассовой классификации
# CrossEntropyLoss = LogSoftmax + NLLLoss
criterion = nn.CrossEntropyLoss()

# Оптимизатор Adam - адаптивный градиентный спуск
# lr=0.001 - стандартное значение для начала
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Scheduler для уменьшения learning rate
# Каждые 5 эпох LR умножается на gamma=0.5
scheduler = torch.optim.lr_scheduler.StepLR(
    optimizer,
    step_size=5,
    gamma=0.5
)

print("Настройка обучения завершена")
print(f"Optimizer: Adam, LR={optimizer.param_groups[0]['lr']}")
print(f"Scheduler: StepLR (step=5, gamma=0.5)")
```

Ожидаемый результат:

- Модель перемещена на GPU (если доступен)
- Настроены: функция потерь CrossEntropyLoss, оптимизатор Adam (lr=0.001), scheduler StepLR

Шаг 5. Функции обучения и валидации

```
from tqdm import tqdm

def train_epoch(model, loader, criterion, optimizer, device):
    """
    Обучение модели на одной эпохе.

    Args:
        model: нейронная сеть
        loader: DataLoader с обучающими данными
        criterion: функция потерь
        optimizer: оптимизатор
        device: устройство (cuda/cpu)

    Returns:
        epoch_loss: средняя потеря за эпоху
        epoch_acc: точность в процентах
    """
    model.train()  # Режим обучения (активирует Dropout)
    running_loss = 0.0
    correct = 0
```

```python
    total = 0

    for images, labels in tqdm(loader, desc='Training'):
        images, labels = images.to(device), labels.to(device)

        # 1. Обнуляем градиенты (важно!)
        optimizer.zero_grad()

        # 2. Forward pass - вычисляем предсказания
        outputs = model(images)

        # 3. Вычисляем функцию потерь
        loss = criterion(outputs, labels)

        # 4. Backward pass - вычисляем градиенты
        loss.backward()

        # 5. Обновляем веса
        optimizer.step()

        # Статистика
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(loader)
    epoch_acc = 100 * correct / total

    return epoch_loss, epoch_acc


def validate_epoch(model, loader, criterion, device):
    """
    Валидация модели на тестовых данных.

    В отличие от обучения:
    - model.eval() - отключает Dropout
    - torch.no_grad() - не вычисляет градиенты
    """
    model.eval()  # Режим оценки (отключает Dropout)
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():  # Не вычисляем градиенты - экономим память
        for images, labels in tqdm(loader, desc='Validating'):
            images, labels = images.to(device), labels.to(device)

            # Только forward pass
            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(loader)
    epoch_acc = 100 * correct / total

    return epoch_loss, epoch_acc


print("Функции обучения и валидации определены")
```

Ожидаемый результат:

- Определены функции `train_epoch()` и `validate_epoch()`
- Функции возвращают среднюю потерю и точность за эпоху

Шаг 6. Цикл обучения

```python
import matplotlib.pyplot as plt
```

```python

# Хранение истории обучения для визуализации
history = {
    'train_loss': [],
    'train_acc': [],
    'val_loss': [],
    'val_acc': []
}

# Параметры обучения
num_epochs = 15
best_acc = 0.0

print(f"Starting training on {device}...")
print("=" * 50)

for epoch in range(num_epochs):
    print(f"\nEpoch {epoch+1}/{num_epochs}")

    # Обучение
    train_loss, train_acc = train_epoch(
        model, train_loader, criterion, optimizer, device
    )

    # Валидация
    val_loss, val_acc = validate_epoch(
        model, test_loader, criterion, device
    )

    # Обновление learning rate
    scheduler.step()

    # Сохранение истории
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)

    # Сохранение лучшей модели
    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), 'best_model.pth')
        print(f"  [*] New best model saved! Accuracy: {val_acc:.2f}%")

    # Вывод статистики
    print(f"  Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
    print(f"  Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%")
    print(f"  Current LR: {optimizer.param_groups[0]['lr']:.6f}")

print("=" * 50)
print(f"\nBest Validation Accuracy: {best_acc:.2f}%")
```

Ожидаемый результат:

- Обучение в течение 15 эпох (на GPU занимает 5–10 минут)
- Train Accuracy должна расти от 80% до 95%
- Val Accuracy должна расти от 85% до 90%
- Лучший результат (best_model.pth) сохраняется автоматически

Признаки успешного обучения:

- Train loss монотонно убывает
- Val loss убывает, затем может стабилизироваться
- Разница между train и val accuracy $< 5\%$ (нет переобучения)

Шаг 7. Визуализация результатов

```python
import matplotlib.pyplot as plt

# Графики обучения
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# График потерь
axes[0].plot(history['train_loss'], label='Train Loss', marker='o')
axes[0].plot(history['val_loss'], label='Val Loss', marker='s')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Кривые потерь')
axes[0].legend()
axes[0].grid(True)

# График точности
axes[1].plot(history['train_acc'], label='Train Acc', marker='o')
axes[1].plot(history['val_acc'], label='Val Acc', marker='s')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy (%)')
axes[1].set_title('Кривые точности')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150)
plt.show()

print("Графики сохранены в training_curves.png")
```

Ожидаемый результат:

- Два графика: потери и точность по эпохам
- Train и Val кривые должны быть близки (нет сильного переобучения)
- Val accuracy должна достигать >85% к концу обучения

Шаг 8. Визуализация предсказаний

```python
# Загрузка лучшей модели
model.load_state_dict(torch.load('best_model.pth'))
model.eval()

# Названия классов FashionMNIST
classes = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat',
           'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Получение батча тестовых данных
images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)

# Предсказания модели
with torch.no_grad():
    outputs = model(images)
    _, predicted = torch.max(outputs, 1)

# Визуализация 10 изображений с предсказаниями
fig, axes = plt.subplots(2, 5, figsize=(15, 6))

for i, ax in enumerate(axes.flat):
    if i < 10:
        # Обратная нормализация для отображения
        img = images[i].cpu().numpy().squeeze()
        img = img * 0.5 + 0.5  # Из [-1,1] в [0,1]

        ax.imshow(img, cmap='gray')

        true_label = classes[labels[i].item()]
        pred_label = classes[predicted[i].item()]

```

```
        # Зелёный если верно, красный если ошибка
        color = 'green' if labels[i] == predicted[i] else 'red'
        ax.set_title(f'Истина: {true_label}\nПредсказание: {pred_label}',
                     color=color, fontsize=9)
        ax.axis('off')

plt.tight_layout()
plt.savefig('predictions.png', dpi=150)
plt.show()

# Подсчёт точности на показанных примерах
correct = (predicted[:10] == labels[:10]).sum().item()
print(f"Точность на 10 примерах: {correct}/10 = {100*correct/10:.0f}%")
```

Ожидаемый результат:

- 10 изображений с подписями (истина vs предсказание)
- Правильные предсказания выделены зелёным, ошибки — красным
- Типичные ошибки: между T-shirt и Shirt, Pullover и Coat

Шаг 9. Визуализация карт признаков

```
def visualize_feature_maps(model, image, layer_name='conv1'):
    """
    Визуализация карт признаков (feature maps) свёрточного слоя.

    Показывает, какие паттерны выделяет каждый фильтр.
    """
    model.eval()

    # Hook для получения активаций
    activations = {}

    def get_activation(name):
        def hook(model, input, output):
            activations[name] = output.detach()
        return hook

    # Регистрация hook на указанный слой
    getattr(model, layer_name).register_forward_hook(
        get_activation(layer_name)
    )

    # Forward pass
    with torch.no_grad():
        model(image.unsqueeze(0).to(device))

    # Получение активаций
    feature_maps = activations[layer_name].cpu().squeeze()

    # Визуализация
    num_maps = min(feature_maps.shape[0], 32)  # Максимум 32 карты
    rows = 4
    cols = 8

    fig, axes = plt.subplots(rows, cols, figsize=(14, 7))

    for i, ax in enumerate(axes.flat):
        if i < num_maps:
            ax.imshow(feature_maps[i], cmap='viridis')
            ax.set_title(f'Фильтр {i}', fontsize=8)
        ax.axis('off')

    plt.suptitle(f'Карты признаков слоя {layer_name}', fontsize=14)
    plt.tight_layout()
    plt.savefig(f'feature_maps_{layer_name}.png', dpi=150)
    plt.show()

    print(f"Размер карт признаков: {feature_maps.shape}")

```

```python

# Визуализация карт первого свёрточного слоя
sample_image, _ = test_dataset[0]
visualize_feature_maps(model, sample_image, 'conv1')
```

Ожидаемый результат:

- 32 карты признаков первого свёрточного слоя (размер 14x14)
- Разные фильтры выделяют разные паттерны: края, углы, текстуры
- Яркие области — высокая активация (фильтр «нашёл» паттерн)

Домашнее задание

1. Дообучить модель до accuracy $> 88\%$ на валидации.
   - Попробовать увеличить количество эпох
   - Экспериментировать с размером батча (32, 128)
   - Изменить архитектуру (добавить слои, изменить количество каналов)
2. Поэкспериментировать с гиперпараметрами:
   - Learning rate: попробовать 0.01, 0.0001
   - Оптимизатор: сравнить Adam и SGD с momentum
3. Добавить сохранение истории обучения в CSV-файл.
4. Подготовить краткий отчёт (Markdown-ячейки в ноутбуке):
   - Описание изменений в архитектуре
   - Таблица с метриками для разных конфигураций
   - Выводы о влиянии параметров на качество

Контрольные вопросы

1. Объясните разницу между Conv2d и Linear слоями.

   Ответ: Conv2d применяет свёртку с разделяемыми весами (фильтры), что значительно уменьшает количество параметров и обеспечивает трансляционную инвариантность. Linear — полносвязный слой, где каждый вход соединён с каждым выходом отдельным весом.

2. Зачем нужна нормализация входных данных?

   Ответ: Нормализация ускоряет сходимость обучения, обеспечивая одинаковый масштаб всех признаков. Без нормализации градиенты могут быть слишком большими или маленькими, что затрудняет оптимизацию.

3. Как работает механизм backpropagation?

   Ответ: Backpropagation вычисляет градиенты функции потерь по всем параметрам сети, используя цепное правило дифференцирования. Градиенты распространяются от выхода к входу, позволяя определить вклад каждого параметра в ошибку.

4. Почему используем Dropout в полносвязных слоях?

   Ответ: Dropout случайно обнуляет часть нейронов во время обучения, что предотвращает коадаптацию признаков и уменьшает переобучение. Модель учится быть более робастной.

5. Как интерпретировать графики train/val loss?

   Ответ: Если train loss « val loss и разрыв растёт — модель переобучается. Если оба высоки — модель недообучена (нужно усложнить архитектуру или обучать дольше). Идеально — обе кривые убывают и сближаются.

Критерии сдачи занятия

- Ноутбук запускается без ошибок (Runtime → Run all).
- Присутствуют все графики (loss curves, predictions, feature maps).
- Accuracy модели на валидации $\geq 85\%$.
- Есть текстовые выводы в Markdown-ячейках.
- Ноутбук загружен в систему до дедлайна.

Занятие 2. Переобучение и регуляризация

Цель занятия

- Наглядно продемонстрировать эффект переобучения (overfitting).
- Изучить методы регуляризации: Dropout, BatchNorm, L2.
- Освоить аугментацию данных через torchvision.transforms.
- Сравнить эффективность методов на практике.

Теоретическая часть (1 час)

Проблема переобучения

Переобучение возникает, когда модель запоминает шум обучающей выборки вместо общих закономерностей. Признаки переобучения:

- `train_loss` $\to 0$, `val_loss` $\to \infty$ (растёт)
- `train_acc` $\to 100\%$, `val_acc` stagnates или падает
- Большой разрыв между метриками на train и val

Методы регуляризации

1. **L2 Regularization (Weight Decay):** Добавление штрафа $\lambda \sum w_i^2$ в функцию потерь. Большие веса штрафуются, модель становится более простой.
2. **Dropout:** Случайное обнуление части нейронов (вероятность p) во время обучения. При инференсе все нейроны активны, но выходы масштабируются.
3. **Batch Normalization:** Нормализация активаций по батчу: $\hat{x} = \frac{x-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}$. Стабилизирует обучение, позволяет использовать большие LR.
4. **Data Augmentation:** Искусственное расширение датасета путём трансформаций: повороты, сдвиги, отражения, изменение яркости.

Практическая часть (3 часа)

Шаг 1. Подготовка урезанного датасета

Для демонстрации переобучения используем только 5% данных.

```python
import torch
from torchvision import datasets, transforms

# Стандартные трансформации
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Полный датасет
full_dataset = datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform
)

# Берём только 5% данных для демонстрации переобучения
subset_size = int(len(full_dataset) * 0.05)  # 3000 примеров
indices = torch.randperm(len(full_dataset))[:subset_size]
small_dataset = torch.utils.data.data.Subset(full_dataset, indices)

# Разделение на train/val
train_size = int(0.8 * len(small_dataset))
val_size = len(small_dataset) - train_size
train_subset, val_subset = torch.utils.data.random_split(
    small_dataset, [train_size, val_size]
)

# DataLoader
train_loader = torch.utils.data.DataLoader(
    train_subset, batch_size=64, shuffle=True, num_workers=0
)
val_loader = torch.utils.data.DataLoader(
    val_subset, batch_size=64, shuffle=False, num_workers=0
)

print(f"Total samples: {len(full_dataset)}")      # 60000
print(f"Subset samples: {len(small_dataset)}")    # 3000
print(f"Train samples: {len(train_subset)}")      # 2400
print(f"Val samples: {len(val_subset)}")          # 600
```

Ожидаемый результат:

- Создана урезанная выборка из 3000 примеров (5% от полной)
- Разделение на train (2400) и val (600)
- Малый размер данных создаст условия для переобучения

Шаг 2. Базовая модель (без регуляризации)

```python
import torch.nn as nn
import torch.nn.functional as F

class OverfitNet(nn.Module):
    """
    Модель с большой ёмкостью для демонстрации переобучения.
    Многие параметры относительно малого количества данных.
    """
    def __init__(self):
        super(OverfitNet, self).__init__()
        # Большая модель для малого датасета -> переобучение
        self.conv1 = nn.Conv2d(1, 64, 3, padding=1)
        self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
        self.fc1 = nn.Linear(128 * 7 * 7, 512)
        self.fc2 = nn.Linear(512, 10)
```

```
        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))  # 28 -> 14
        x = self.pool(F.relu(self.conv2(x)))  # 14 -> 7
        x = x.view(-1, 128 * 7 * 7)
        x = F.relu(self.fc1(x))  # Без Dropout!
        x = self.fc2(x)
        return x

model = OverfitNet()

# Подсчёт параметров
total_params = sum(p.numel() for p in model.parameters())
print(f"Параметров модели: {total_params:,}")  # ~1.7 млн
print(f"Параметров на пример: {total_params/len(train_subset):.0f}")
# ~700 параметров на пример -> переобучение вероятно
```

Ожидаемый результат:

- Модель с 1.7 млн параметров
- На каждый пример обучающей выборки приходится 700 параметров
- Это типичная ситуация, ведущая к переобучению

Шаг 3. Эксперимент 1: Демонстрация переобучения

```
# Обучаем на малой выборке без регуляризации
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = OverfitNet().to(device)

criterion = nn.CrossEntropyLoss()
# weight_decay=0 означает отсутствие L2 регуляризации
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, weight_decay=0.0)

# История для сохранения результатов
history_baseline = {'train_loss': [], 'train_acc': [],
                    'val_loss': [], 'val_acc': []}

num_epochs = 30  # Много эпох для проявления переобучения

for epoch in range(num_epochs):
    # Обучение (функция из занятия 1)
    train_loss, train_acc = train_epoch(
        model, train_loader, criterion, optimizer, device
    )
    # Валидация
    val_loss, val_acc = validate_epoch(
        model, val_loader, criterion, device
    )

    history_baseline['train_loss'].append(train_loss)
    history_baseline['train_acc'].append(train_acc)
    history_baseline['val_loss'].append(val_loss)
    history_baseline['val_acc'].append(val_acc)

    if epoch % 5 == 0 or epoch == num_epochs - 1:
        print(f"Epoch {epoch+1}: Train Acc={train_acc:.1f}%, "
              f"Val Acc={val_acc:.1f}%, Gap={train_acc-val_acc:.1f}%")

# Анализ результатов
gap = history_baseline['train_acc'][-1] - history_baseline['val_acc'][-1]
print(f"\nИтоговый разрыв (Overfitting Gap): {gap:.1f}%")
# Ожидается: Train Acc ~99%, Val Acc ~60%, Gap ~39%
```

Ожидаемый результат:

- **Train Accuracy:** растёт до 95–99% (модель «запоминает» данные)

- **Val Accuracy:** сначала растёт, потом падает или стагнирует (55–65%)
- **Overfitting Gap:** разрыв между train и val accuracy 30–40%

Интерпретация: Большой разрыв указывает на переобучение — модель запомнила обучающие примеры, но не обобщает на новые данные.

Шаг 4. Эксперимент 2: Dropout и Weight Decay

```
class RegularizedNet(nn.Module):
    """
    Модель с регуляризацией: BatchNorm и Dropout.
    """
    def __init__(self):
        super(RegularizedNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 64, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(64)  # Batch Normalization
        self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(128)
        self.fc1 = nn.Linear(128 * 7 * 7, 512)
        self.dropout = nn.Dropout(0.5)  # Dropout с вероятностью 0.5
        self.fc2 = nn.Linear(512, 10)
        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        # BatchNorm применяется после conv, до активации
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = x.view(-1, 128 * 7 * 7)
        x = self.dropout(F.relu(self.fc1(x)))  # Dropout после активации
        x = self.fc2(x)
        return x

model_reg = RegularizedNet().to(device)

# Оптимизатор с L2 регуляризацией (weight_decay)
optimizer = torch.optim.SGD(
    model_reg.parameters(),
    lr=0.01,
    weight_decay=1e-4  # L2 penalty = 0.0001
)

history_reg = {'train_loss': [], 'train_acc': [],
               'val_loss': [], 'val_acc': []}

for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(
        model_reg, train_loader, criterion, optimizer, device
    )
    val_loss, val_acc = validate_epoch(
        model_reg, val_loader, criterion, device
    )

    history_reg['train_loss'].append(train_loss)
    history_reg['train_acc'].append(train_acc)
    history_reg['val_loss'].append(val_loss)
    history_reg['val_acc'].append(val_acc)

gap_reg = history_reg['train_acc'][-1] - history_reg['val_acc'][-1]
print(f"\nС регуляризацией: Train={history_reg['train_acc'][-1]:.1f}%, "
      f"Val={history_reg['val_acc'][-1]:.1f}%, Gap={gap_reg:.1f}%")
# Ожидается: Train ~90-95%, Val ~80-85%, Gap ~10%
```

Ожидаемый результат:

- **Train Accuracy:** ниже, чем без регуляризации (90–95%)
- **Val Accuracy:** значительно выше (80–85%)
- **Overfitting Gap:** уменьшился до 10%

Сравнение методов регуляризации:

Метод	Train Acc	Val Acc	Gap
Без регуляризации	99%	60%	39%
Dropout + BN + L2	92%	85%	7%

Таблица 5: Сравнение методов регуляризации

Шаг 5. Эксперимент 3: Аугментация данных

```
# Трансформации с аугментацией данных
train_transform_aug = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),  # Случайное отражение
    transforms.RandomRotation(10),           # Поворот на ±10 градусов
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Перезагрузка датасета с аугментацией
aug_dataset = datasets.FashionMNIST(
    root='./data', train=True, download=True,
    transform=train_transform_aug
)

# Применяем те же индексы подмножества
aug_subset = torch.utils.data.Subset(aug_dataset, indices)
train_aug, val_aug = torch.utils.data.random_split(
    aug_subset, [train_size, val_size]
)

train_loader_aug = torch.utils.data.DataLoader(
    train_aug, batch_size=64, shuffle=True, num_workers=0
)

# Обучаем модель с аугментацией
model_aug = RegularizedNet().to(device)
optimizer = torch.optim.SGD(model_aug.parameters(), lr=0.01, weight_decay=1e-4)

history_aug = {'train_loss': [], 'train_acc': [],
               'val_loss': [], 'val_acc': []}

for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(
        model_aug, train_loader_aug, criterion, optimizer, device
    )
    val_loss, val_acc = validate_epoch(
        model_aug, val_loader, criterion, device  # Val без аугментации!
    )

    history_aug['train_loss'].append(train_loss)
    history_aug['train_acc'].append(train_acc)
    history_aug['val_loss'].append(val_loss)
    history_aug['val_acc'].append(val_acc)

gap_aug = history_aug['train_acc'][-1] - history_aug['val_acc'][-1]
print(f"\nС аугментацией: Train={history_aug['train_acc'][-1]:.1f}%, "
      f"Val={history_aug['val_acc'][-1]:.1f}%, Gap={gap_aug:.1f}%")
# Ожидается: Val Acc выше, Gap ещё меньше
```

Ожидаемый результат:

- **Train Accuracy:** может быть ниже (аугментация «создаёт» новые примеры)
- **Val Accuracy:** ещё выше (85–90%)
- **Overfitting Gap:** минимальный (5–8%)

Шаг 6. Сравнение результатов

```
import matplotlib.pyplot as plt
```

```python
import pandas as pd

# Сводная таблица результатов
results = {
    'Эксперимент': ['Baseline (переобучение)', 'Dropout + L2 + BN',
                    'Аугментация'],
    'Train Acc (%)': [history_baseline['train_acc'][-1],
                      history_reg['train_acc'][-1],
                      history_aug['train_acc'][-1]],
    'Val Acc (%)': [history_baseline['val_acc'][-1],
                    history_reg['val_acc'][-1],
                    history_aug['val_acc'][-1]],
    'Gap (%)': [history_baseline['train_acc'][-1] - history_baseline['val_acc'][-1],
                history_reg['train_acc'][-1] - history_reg['val_acc'][-1],
                history_aug['train_acc'][-1] - history_aug['val_acc'][-1]]
}

df = pd.DataFrame(results)
print(df.to_string(index=False))

# Визуализация сравнения
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# График Val Accuracy для всех экспериментов
axes[0].plot(history_baseline['val_acc'], label='Baseline', linestyle='--')
axes[0].plot(history_reg['val_acc'], label='Dropout + L2 + BN')
axes[0].plot(history_aug['val_acc'], label='Аугментация')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Validation Accuracy (%)')
axes[0].set_title('Сравнение Val Accuracy')
axes[0].legend()
axes[0].grid(True)

# График Gap (переобучения)
gaps = [results['Gap (%)'][i] for i in range(3)]
axes[1].bar(results['Эксперимент'], results['Val Acc (%)'],
            color=['red', 'orange', 'green'], alpha=0.7)
axes[1].set_ylabel('Validation Accuracy (%)')
axes[1].set_title('Итоговое сравнение методов')
axes[1].set_ylim([0, 100])

for i, v in enumerate(results['Val Acc (%)']):
    axes[1].text(i, v + 2, f'{v:.1f}%', ha='center')

plt.tight_layout()
plt.savefig('regularization_comparison.png', dpi=150)
plt.show()
```

Ожидаемые результаты сравнения:

Эксперимент	Train Acc	Val Acc	Gap
Baseline (переобучение)	99.5%	62.0%	37.5%
Dropout + L2 + BN	95.0%	85.0%	10.0%
Аугментация	93.0%	88.0%	5.0%

Таблица 6: Ожидаемые результаты экспериментов

Выводы:

- L2 регуляризация штрафует большие веса, упрощая модель
- Dropout предотвращает коадаптацию нейронов
- BatchNorm стабилизирует обучение и ускоряет сходимость
- Аугментация данных увеличивает эффективный размер выборки
- Комбинация методов даёт наилучший результат

Домашнее задание

1. Подобрать оптимальный коэффициент weight_decay из ряда $10^{-3}, 10^{-4}, 10^{-5}$.
   - Слишком большой вес (10^{-3}) может недообучить модель
   - Слишком маленький (10^{-5}) не даст эффекта
   - Оптимальный обычно в диапазоне 10^{-4} – 10^{-3}
2. Попробовать комбинацию всех методов (BN + Dropout + Aug + L2).
3. Исследовать влияние вероятности Dropout (0.1, 0.3, 0.5, 0.7).
4. Заполнить таблицу результатов в отчёте с выводами.

Контрольные вопросы

1. Почему accuracy на обучении может быть 100%, а на валидации низкой?

   Ответ: Модель запомнила обучающие примеры вместе с шумом, но не выучила общие закономерности. Это классическое переобучение.

2. Как работает Dropout во время обучения и во время инференса?

   Ответ: Во время обучения Dropout случайно обнуляет нейроны с вероятностью p. Во время инференса все нейроны активны, но их выходы умножаются на $(1-p)$ для сохранения масштаба активаций.

3. Зачем нужна Batch Normalization перед активацией или после?

   Ответ: Обычно BN применяют после свёртки и до активации. Это нормализует входные данные для функции активации, предотвращая затухание или взрыв градиентов. Некоторые исследования показывают, что BN после активации тоже работает.

4. Как аугментация влияет на размер эффективного датасета?

   Ответ: Аугментация создаёт «новые» примеры на основе существующих. Например, RandomHorizontalFlip удваивает эффективный размер для симметричных классов. Комбинация трансформаций может увеличить эффективный размер в 10–100 раз.

Занятие 3. Transfer Learning: Feature Extraction vs Fine-tuning

Цель занятия

- Освоить работу с предобученными моделями (torchvision.models).
- Изучить стратегии Transfer Learning: Feature Extraction и Fine-tuning.
- Решить задачу классификации на малом датасете.
- Сравнить эффективность двух стратегий.

Теоретическая часть (1 час)

Идея Transfer Learning

Transfer Learning (перенос обучения) — использование весов, обученных на большом датасете (например, ImageNet с 1.2M изображений), для решения новой задачи с малым количеством данных. Это позволяет:

- Получить высокое качество при малом количестве данных
- Значительно сократить время обучения
- Использовать признаки, выученные на похожих задачах

Две основные стратегии:

1. **Feature Extraction (Freeze):** Заморозка всех свёрточных слоёв, обучение только классификатора.
   - Быстрое обучение (мало параметров)
   - Меньший риск переобучения
   - Подходит для очень малых датасетов (<1000 примеров)
2. **Fine-tuning:** Разморозка части или всех слоёв, дообучение с малым LR.
   - Лучшая адаптация к новой задаче
   - Требует больше данных для избежания переобучения
   - Более длительное обучение

Архитектура ResNet

ResNet использует остаточные связи (skip connections): $y = F(x) + x$. Это позволяет обучать очень глубокие сети (18, 50, 101, 152 слоя) без проблемы затухания градиента. Для практикума используем ResNet18 — хороший баланс между качеством и скоростью.

Практическая часть (3 часа)

Шаг 1. Загрузка датасета

```python
import os
import torch
from torchvision import datasets, transforms, models
import torch.nn as nn

# Трансформации для Transfer Learning
# Важно: использовать нормализацию ImageNet!
data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),      # Случайный кадр 224x224
        transforms.RandomHorizontalFlip(),      # Аугментация: отражение
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],  # Mean ImageNet
                             [0.229, 0.224, 0.225])  # Std ImageNet
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),                 # Маштабирование
        transforms.CenterCrop(224),             # Центральный кадр
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225])
    ]),
}

# Попытка загрузки датасета Hymenoptera (ants vs bees)
# Если недоступен - используем CIFAR-10 как fallback
data_dir = './data/hymenoptera_data'

try:
    # Пробуем загрузить Hymenoptera
    image_datasets = {x: datasets.ImageFolder(
        os.path.join(data_dir, x), data_transforms[x]
    ) for x in ['train', 'val']}

    dataloaders = {x: torch.utils.data.DataLoader(
        image_datasets[x], batch_size=4, shuffle=True, num_workers=0
    ) for x in ['train', 'val']}

    dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
    class_names = image_datasets['train'].classes
    print(f"Загружен датасет Hymenoptera")
    print(f"Классы: {class_names}")  # ['ants', 'bees']

except (FileNotFoundError, RuntimeError):
    # Fallback: используем подмножество CIFAR-10
    print("Hymenoptera недоступен, используем CIFAR-10")

    # CIFAR-10 нужно масштабировать до 224x224 для ResNet
    cifar_transform = transforms.Compose([
        transforms.Resize(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406],
                             [0.229, 0.224, 0.225])
    ])

    cifar_train = datasets.CIFAR10(
        root='./data', train=True, download=True, transform=cifar_transform
    )
    cifar_val = datasets.CIFAR10(
        root='./data', train=False, download=True, transform=cifar_transform
    )

    # Берём только 2 класса (cat, dog) и 500 примеров каждого
```

```
    def filter_dataset(dataset, classes=[3, 5], samples_per_class=500):
        indices = []
        for cls in classes:
            cls_indices = (torch.tensor(dataset.targets) == cls).nonzero()
            indices.extend(cls_indices[:samples_per_class].tolist())
        return torch.utils.data.Subset(dataset, indices)

    train_subset = filter_dataset(cifar_train)
    val_subset = filter_dataset(cifar_val, samples_per_class=100)

    dataloaders = {
        'train': torch.utils.data.DataLoader(
            train_subset, batch_size=16, shuffle=True, num_workers=0
        ),
        'val': torch.utils.data.DataLoader(
            val_subset, batch_size=16, shuffle=False, num_workers=0
        )
    }

    dataset_sizes = {'train': len(train_subset), 'val': len(val_subset)}
    class_names = ['cat', 'dog']

    print(f"Используем подмножество CIFAR-10: {class_names}")
    print(f"Train: {dataset_sizes['train']}, Val: {dataset_sizes['val']}")

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Ожидаемый результат:

- Загружен датасет Hymenoptera (ants vs bees) или CIFAR-10 fallback
- Данные подготовлены с нормализацией ImageNet
- Классы: ['ants', 'bees'] или ['cat', 'dog']

Шаг 2. Загрузка предобученной модели

```
# Загрузка предобученной ResNet18
model_ft = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# Вывод архитектуры
print(model_ft)

# Подсчёт параметров
total_params = sum(p.numel() for p in model_ft.parameters())
trainable_params = sum(p.numel() for p in model_ft.parameters() if p.requires_grad)

print(f"Всего параметров: {total_params:,}")
print(f"Обучаемых: {trainable_params:,}")

# Последний слой (fc) нужно заменить под нашу задачу
num_ftrs = model_ft.fc.in_features  # 512 для ResNet18
print(f"Размер входа fc слоя: {num_ftrs}")
```

Ожидаемый результат:

- Загружена ResNet18 с весами ImageNet
- Всего параметров: 11.7 млн
- Последний fc слой ожидает 512 признаков на входе

Шаг 3. Стратегия 1: Feature Extraction (Freeze)

```python
# Создаём копию модели для Feature Extraction
model_fe = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# Замораживаем ВСЕ параметры backbone
for param in model_fe.parameters():
    param.requires_grad = False

# Заменяем последний слой под нашу задачу (2 класса)
num_ftrs = model_fe.fc.in_features
model_fe.fc = nn.Linear(num_ftrs, 2)  # Только этот слой будет обучаться!
model_fe = model_fe.to(device)

# Подсчёт обучаемых параметров
trainable_fe = sum(p.numel() for p in model_fe.parameters() if p.requires_grad)
frozen_fe = sum(p.numel() for p in model_fe.parameters() if not p.requires_grad)

print("=== Feature Extraction (Freeze) ===")
print(f"Обучаемых параметров: {trainable_fe:,}")      # ~1024
print(f"Замороженных параметров: {frozen_fe:,}")      # ~11.7M
print(f"Процент обучаемых: {100*trainable_fe/total_params:.2f}%")

# Оптимизатор только для fc слоя
optimizer_fe = torch.optim.SGD(model_fe.fc.parameters(), lr=0.01, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

Ожидаемый результат:

- Заморожено 11.7 млн параметров (backbone)
- Обучается только fc слой (1024 параметра)
- Процент обучаемых параметров: <0.01%

Шаг 4. Обучение (Feature Extraction)

```python
from tqdm import tqdm
import copy

def train_model(model, dataloaders, criterion, optimizer, num_epochs=10):
    """Обучение модели с сохранением лучшей версии"""
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}

    for epoch in range(num_epochs):
        print(f'Epoch {epoch+1}/{num_epochs}')

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in tqdm(dataloaders[phase], desc=phase):
                inputs, labels = inputs.to(device), labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

```

```
                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = running_corrects.double() / dataset_sizes[phase]

            if phase == 'train':
                history['train_loss'].append(epoch_loss)
                history['train_acc'].append(epoch_acc.item() * 100)
            else:
                history['val_loss'].append(epoch_loss)
                history['val_acc'].append(epoch_acc.item() * 100)

                if epoch_acc > best_acc:
                    best_acc = epoch_acc
                    best_model_wts = copy.deepcopy(model.state_dict())

        print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

    model.load_state_dict(best_model_wts)
    return model, history, best_acc

# Обучение Feature Extraction
print("\nОбучение Feature Extraction...")
model_fe, history_fe, best_acc_fe = train_model(
    model_fe, dataloaders, criterion, optimizer_fe, num_epochs=10
)
print(f"\nЛучший результат Feature Extraction: {best_acc_fe*100:.2f}%")
```

Ожидаемый результат для Feature Extraction:

- Быстрая сходимость (5–10 эпох достаточно)
- Val Accuracy: 90–94% (зависит от датасета)
- Время обучения: 2–5 минут на GPU

Шаг 5. Стратегия 2: Fine-tuning

```
# Создаём новую модель для Fine-tuning
model_ft = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# Заменяем последний слой
num_ftrs = model_ft.fc.in_features
model_ft.fc = nn.Linear(num_ftrs, 2)
model_ft = model_ft.to(device)

# Размораживаем последний conv блок (layer4)
for param in model_ft.parameters():
    param.requires_grad = False

# Размораживаем layer4
for param in model_ft.layer4.parameters():
    param.requires_grad = True

# И fc слой тоже должен быть обучаемым
for param in model_ft.fc.parameters():
    param.requires_grad = True

# Подсчёт параметров
trainable_ft = sum(p.numel() for p in model_ft.parameters() if p.requires_grad)
print("=== Fine-tuning ===")
print(f"Обучаемых параметров: {trainable_ft:,}")  # ~8.5M

# Оптимизатор с разными LR для разных частей
optimizer_ft = torch.optim.SGD([
    {'params': model_ft.layer4.parameters(), 'lr': 0.001},  # Малый LR для pretrained
    {'params': model_ft.fc.parameters(), 'lr': 0.01}        # Большой LR для нового
], momentum=0.9)

print("\nОбучение Fine-tuning...")
model_ft, history_ft, best_acc_ft = train_model(
```

```
    model_ft, dataloaders, criterion, optimizer_ft, num_epochs=10
)
print(f"\nЛучший результат Fine-tuning: {best_acc_ft*100:.2f}%")
```

Ожидаемый результат для Fine-tuning:

- Val Accuracy: 93–97% (выше, чем Feature Extraction)
- Время обучения: 5–10 минут (дольше из-за большего количества параметров)
- Риск переобучения выше при малых датасетах

Шаг 6. Сравнение стратегий

```
import matplotlib.pyplot as plt
import pandas as pd

# Сводная таблица
comparison = {
    'Стратегия': ['Feature Extraction', 'Fine-tuning'],
    'Обучаемых параметров': [1024, 8500000],
    'Время/эпоха': ['~10 сек', '~30 сек'],
    'Val Accuracy': [f'{best_acc_fe*100:.1f}%', f'{best_acc_ft*100:.1f}%'],
    'Риск переобучения': ['Низкий', 'Средний']
}

df = pd.DataFrame(comparison)
print("\n" + "="*60)
print(df.to_string(index=False))
print("="*60)

# Визуализация кривых обучения
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Val Accuracy
axes[0].plot(history_fe['val_acc'], label='Feature Extraction', marker='o')
axes[0].plot(history_ft['val_acc'], label='Fine-tuning', marker='s')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Validation Accuracy (%)')
axes[0].set_title('Сравнение Val Accuracy')
axes[0].legend()
axes[0].grid(True)

# Val Loss
axes[1].plot(history_fe['val_loss'], label='Feature Extraction', marker='o')
axes[1].plot(history_ft['val_loss'], label='Fine-tuning', marker='s')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Validation Loss')
axes[1].set_title('Сравнение Val Loss')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.savefig('transfer_learning_comparison.png', dpi=150)
plt.show()
```

Сравнительный анализ:
Рекомендации по выбору стратегии:

- **Feature Extraction:** малый датасет (<500 примеров), ограниченное время
- **Fine-tuning:** достаточный датасет (>500 примеров), нужна максимальная точность

Метрика	Feature Extraction	Fine-tuning
Обучаемых параметров	∼1K	∼8.5M
Время на эпоху	∼10 сек	∼30 сек
Ожидаемая Val Acc	90–94%	93–97%
Риск переобучения	Низкий	Средний
Требования к данным	>100 примеров	>500 примеров

Таблица 7: Сравнение стратегий Transfer Learning

Домашнее задание

1. Попробовать разморозить все слои сети (Full Fine-tuning).
2. Сравнить время обучения и итоговую точность с partial Fine-tuning.
3. Экспериментировать с разными значениями LR для fine-tuning.
4. Сохранить веса лучшей модели (.pth файл).

Контрольные вопросы

1. Почему важно использовать нормализацию ImageNet для предобученных моделей?

   Ответ: Предобученные модели обучались на данных, нормализованных именно этими значениями mean и std. Использование других значений приведёт к распределению данных, которое модель «не ожидает», и качество упадёт.

2. В чём риск размораживания всех слоёв при малом датасете?

   Ответ: При размораживании всех слоёв модель может «забыть» полезные признаки, выученные на ImageNet, и переобучиться на малом датасете. Это называется catastrophic forgetting.

3. Почему Learning Rate для Fine-tuning должен быть меньше?

   Ответ: Предобученные веса уже близки к оптимальным. Большой LR может слишком сильно изменить их, разрушив полезные признаки. Малый LR позволяет «подстроить» веса под новую задачу, не разрушая уже выученное.

Критерии сдачи занятия

- Реализованы обе стратегии (Freeze и Fine-tune).
- Получена точность на валидации $\geq 90\%$.
- Присутствует сравнение времени обучения и количества параметров.
- Визуализированы кривые обучения для обеих стратегий.

Занятие 4. Оптимизация, Дистилляция и Mutual Learning

Цель занятия

- Изучить влияние оптимизаторов и планировщиков скорости обучения.
- Реализовать дистилляцию знаний (Knowledge Distillation).
- Реализовать взаимное обучение (Deep Mutual Learning).
- Сравнить эффективность сжатия моделей.

Теоретическая часть (1 час)

Оптимизаторы и Scheduler

- **SGD vs Adam:** SGD обобщается лучше, Adam сходится быстрее. Adam адаптивно подстраивает LR для каждого параметра.
- **Learning Rate Scheduler:** Изменение LR в процессе обучения.
  - StepLR: уменьшение LR в заданные эпохи
  - CosineAnnealing: плавное изменение по косинусу
  - ReduceLROnPlateau: уменьшение при стагнации метрики

Дистилляция знаний (Knowledge Distillation)

Обучение маленькой модели (Student) с помощью большой (Teacher). Функция потерь:

$$L = \alpha \cdot CE(S, y) + (1 - \alpha) \cdot T^2 \cdot KL(S_\tau, T_\tau)$$

где:

- $CE(S, y)$ — кросс-энтропия с истинными метками
- $KL(S_\tau, T_\tau)$ — KL-дивергенция между «мягкими» распределениями
- T — температура, смягчающая распределение вероятностей
- α — баланс между hard и soft loss

Взаимное обучение (Deep Mutual Learning)

Две модели (Student A, Student B) обучаются совместно, используя предсказания друг друга как мягкие цели. Не требует предобученного Teacher.

$$L_A = CE(A, y) + KL(A_\tau, B_\tau)$$

Практическая часть (3 часа)

Шаг 1. Подготовка данных и моделей

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms, models

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Трансформации для CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

# Загрузка CIFAR-10
train_dataset = datasets.CIFAR10(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = datasets.CIFAR10(
    root='./data', train=False, download=True, transform=transform
)

# Используем подмножество для ускорения экспериментов
train_subset = torch.utils.data.Subset(train_dataset, range(10000))
test_subset = torch.utils.data.Subset(test_dataset, range(2000))

train_loader = torch.utils.data.DataLoader(
    train_subset, batch_size=128, shuffle=True, num_workers=0
)
test_loader = torch.utils.data.DataLoader(
    test_subset, batch_size=128, shuffle=False, num_workers=0
)

print(f"Train samples: {len(train_subset)}")
print(f"Test samples: {len(test_subset)}")


# Teacher Model - ResNet18 (предобученная)
def create_teacher():
    """Создание Teacher модели на основе ResNet18"""
    teacher = models.resnet18(weights=None)
    # Адаптация под CIFAR-10 (32x32 вместо 224x224)
    teacher.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
    teacher.maxpool = nn.Identity()  # Убираем maxpool для маленьких изображений
    teacher.fc = nn.Linear(512, 10)
    return teacher.to(device)


# Student Model - маленькая CNN
class StudentCNN(nn.Module):
    """Лёгкая модель для дистилляции"""
    def __init__(self):
        super(StudentCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),  # 32 -> 16
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),  # 16 -> 8
            nn.Conv2d(64, 64, 3, padding=1),
            nn.ReLU(),
```

```
            nn.MaxPool2d(2),  # 8 -> 4
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 4 * 4, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)

# Подсчёт параметров
teacher = create_teacher()
student = StudentCNN().to(device)

teacher_params = sum(p.numel() for p in teacher.parameters())
student_params = sum(p.numel() for p in student.parameters())

print(f"Teacher (ResNet18) параметров: {teacher_params:,}")  # ~11M
print(f"Student (SimpleCNN) параметров: {student_params:,}") # ~140K
print(f"Коэффициент сжатия: {teacher_params/student_params:.1f}x")
```

Ожидаемый результат:

- Teacher: 11 млн параметров
- Student: 140 тыс. параметров
- Коэффициент сжатия: 80x

Шаг 2. Обучение Teacher модели

```
def train_teacher(model, train_loader, test_loader, epochs=10):
    """Обучение Teacher модели"""
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
    criterion = nn.CrossEntropyLoss()

    best_acc = 0.0
    for epoch in range(epochs):
        model.train()
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

        # Валидация
        model.eval()
        correct = total = 0
        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        acc = 100 * correct / total
        if acc > best_acc:
            best_acc = acc
            torch.save(model.state_dict(), 'teacher_best.pth')

        print(f"Teacher Epoch {epoch+1}/{epochs}: Test Acc = {acc:.2f}%")

    model.load_state_dict(torch.load('teacher_best.pth'))
    return model, best_acc
```

```python

# Обучаем Teacher
print("=== Обучение Teacher ===")
teacher, teacher_acc = train_teacher(teacher, train_loader, test_loader, epochs=15)
print(f"\nЛучшая точность Teacher: {teacher_acc:.2f}%")
```

Ожидаемый результат:

- Teacher accuracy: 65–75% (на подмножестве CIFAR-10)
- Модель сохранена в teacher_best.pth

Шаг 3. Реализация Distillation Loss

```python
def distillation_loss(student_outputs, teacher_outputs, labels,
                      temperature=4.0, alpha=0.7):
    """
    Вычисление функции потерь для дистилляции.

    Args:
        student_outputs: логиты Student модели
        teacher_outputs: логиты Teacher модели
        labels: истинные метки
        temperature: температура для смягчения распределения
        alpha: вес hard loss (CE с истинными метками)

    Returns:
        combined_loss: комбинированная функция потерь

    Температура T > 1 смягчает распределение вероятностей:
    - При T=1: стандартный softmax
    - При T=4: более «гладкое» распределение

    Это позволяет Student модели выучить не только правильный класс,
    но и относительные сходства между классами (dark knowledge).
    """
    # Hard loss: кросс-энтропия с истинными метками
    ce_loss = nn.CrossEntropyLoss()(student_outputs, labels)

    # Soft loss: KL-дивергенция между мягкими распределениями
    # Делим логиты на T для смягчения
    kd_loss = nn.KLDivLoss(reduction='batchmean')(
        F.log_softmax(student_outputs / temperature, dim=1),
        F.softmax(teacher_outputs / temperature, dim=1)
    ) * (temperature * temperature)  # Масштабируем обратно

    # Комбинированный loss
    return alpha * ce_loss + (1 - alpha) * kd_loss


def train_with_distillation(teacher, student, train_loader, test_loader,
                            temperature=4.0, alpha=0.7, epochs=15):
    """Обучение Student с дистилляцией от Teacher"""
    teacher.eval()  # Teacher заморожен

    optimizer = torch.optim.Adam(student.parameters(), lr=0.001)

    best_acc = 0.0
    history = {'train_loss': [], 'test_acc': []}

    for epoch in range(epochs):
        student.train()
        total_loss = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()

            # Teacher предсказания (без градиентов)
            with torch.no_grad():
                teacher_outputs = teacher(images)

```

```
            # Student предсказания
            student_outputs = student(images)

            # Distillation loss
            loss = distillation_loss(
                student_outputs, teacher_outputs, labels,
                temperature=temperature, alpha=alpha
            )

            loss.backward()
            optimizer.step()
            total_loss += loss.item()

        # Валидация
        student.eval()
        correct = total = 0
        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = student(images)
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        acc = 100 * correct / total
        avg_loss = total_loss / len(train_loader)

        history['train_loss'].append(avg_loss)
        history['test_acc'].append(acc)

        if acc > best_acc:
            best_acc = acc
            torch.save(student.state_dict(), 'student_kd_best.pth')

        print(f"KD Epoch {epoch+1}/{epochs}: Loss={avg_loss:.4f}, Acc={acc:.2f}%")

    student.load_state_dict(torch.load('student_kd_best.pth'))
    return student, best_acc, history


# Обучение Student с дистилляцией
print("\n=== Обучение Student с Knowledge Distillation ===")
student_kd = StudentCNN().to(device)
student_kd, kd_acc, history_kd = train_with_distillation(
    teacher, student_kd, train_loader, test_loader,
    temperature=4.0, alpha=0.7, epochs=15
)
print(f"\nЛучшая точность Student (KD): {kd_acc:.2f}%")
```

Ожидаемый результат:

- Student с KD показывает точность 55–65% (выше, чем без дистилляции)
- Используя знания Teacher, Student «учится» у большой модели

Шаг 4. Реализация Mutual Learning

```
def mutual_learning_loss(out_A, out_B, labels, temperature=4.0):
    """
    Функция потерь для взаимного обучения двух моделей.

    Каждая модель учится у предсказаний другой модели,
    а также у истинных меток.
    """
    # Cross-entropy с истинными метками
    ce_A = nn.CrossEntropyLoss()(out_A, labels)
    ce_B = nn.CrossEntropyLoss()(out_B, labels)

    # KL-дивергенция: A учится у B
    kl_A = nn.KLDivLoss(reduction='batchmean')(
        F.log_softmax(out_A / temperature, dim=1),
        F.softmax(out_B / temperature, dim=1)
    ) * (temperature * temperature)
```

```python

    # KL-дивергенция: B учится у A
    kl_B = nn.KLDivLoss(reduction='batchmean')(
        F.log_softmax(out_B / temperature, dim=1),
        F.softmax(out_A / temperature, dim=1)
    ) * (temperature * temperature)

    # Общий loss - каждая модель минимизирует все компоненты
    return ce_A + ce_B + kl_A + kl_B


def train_mutual_learning(train_loader, test_loader, epochs=15, temperature=4.0):
    """Обучение двух моделей методом Mutual Learning"""

    # Два одинаковых Student-а
    student_A = StudentCNN().to(device)
    student_B = StudentCNN().to(device)

    opt_A = torch.optim.Adam(student_A.parameters(), lr=0.001)
    opt_B = torch.optim.Adam(student_B.parameters(), lr=0.001)

    best_acc_A = best_acc_B = 0.0
    history = {'acc_A': [], 'acc_B': []}

    for epoch in range(epochs):
        student_A.train()
        student_B.train()

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            opt_A.zero_grad()
            opt_B.zero_grad()

            out_A = student_A(images)
            out_B = student_B(images)

            # Mutual learning loss
            loss = mutual_learning_loss(out_A, out_B, labels, temperature)

            loss.backward()
            opt_A.step()
            opt_B.step()

        # Валидация
        student_A.eval()
        student_B.eval()

        correct_A = correct_B = total = 0
        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)

                out_A = student_A(images)
                out_B = student_B(images)

                _, pred_A = torch.max(out_A, 1)
                _, pred_B = torch.max(out_B, 1)

                total += labels.size(0)
                correct_A += (pred_A == labels).sum().item()
                correct_B += (pred_B == labels).sum().item()

        acc_A = 100 * correct_A / total
        acc_B = 100 * correct_B / total

        history['acc_A'].append(acc_A)
        history['acc_B'].append(acc_B)

        best_acc_A = max(best_acc_A, acc_A)
        best_acc_B = max(best_acc_B, acc_B)

        print(f"DML Epoch {epoch+1}/{epochs}: A={acc_A:.2f}%, B={acc_B:.2f}%")

    return student_A, student_B, best_acc_A, best_acc_B, history


# Обучение Mutual Learning
```

```
print("\n=== Deep Mutual Learning ===")
student_A, student_B, acc_A, acc_B, history_dml = train_mutual_learning(
    train_loader, test_loader, epochs=15
)
print(f"\nЛучшие результаты DML: A={acc_A:.2f}%, B={acc_B:.2f}%")
```

Ожидаемый результат:

- Обе модели показывают схожую точность 55–65%
- Mutual Learning не требует предобученного Teacher
- Модели могут быть усреднены (ensemble) для лучшего результата

Шаг 5. Сравнение результатов

```
import matplotlib.pyplot as plt
import pandas as pd

# Обучение baseline Student (без дистилляции)
print("\n=== Обучение Baseline Student ===")
student_baseline = StudentCNN().to(device)
optimizer = torch.optim.Adam(student_baseline.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

history_baseline = {'test_acc': []}
best_baseline = 0.0

for epoch in range(15):
    student_baseline.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        loss = criterion(student_baseline(images), labels)
        loss.backward()
        optimizer.step()

    # Валидация
    student_baseline.eval()
    correct = total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = student_baseline(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    acc = 100 * correct / total
    history_baseline['test_acc'].append(acc)
    best_baseline = max(best_baseline, acc)
    print(f"Baseline Epoch {epoch+1}: Acc={acc:.2f}%")

print(f"\nЛучшая точность Baseline: {best_baseline:.2f}%")


# Итоговое сравнение
print("\n" + "="*60)
print("ИТОГОВОЕ СРАВНЕНИЕ")
print("="*60)

results = {
    'Метод': ['Baseline Student', 'Student + KD', 'DML (Model A)', 'DML (Model B)'],
    'Параметры': [f'{student_params:,}'] * 4,
    'Test Accuracy': [f'{best_baseline:.1f}%', f'{kd_acc:.1f}%',
                      f'{acc_A:.1f}%', f'{acc_B:.1f}%'],
    'Требует Teacher': ['Нет', 'Да', 'Нет', 'Нет']
}

df = pd.DataFrame(results)
print(df.to_string(index=False))

```

```python
# Визуализация
plt.figure(figsize=(10, 5))
plt.plot(history_baseline['test_acc'], label='Baseline', linestyle='--')
plt.plot(history_kd['test_acc'], label='Knowledge Distillation')
plt.plot(history_dml['acc_A'], label='DML Model A')
plt.xlabel('Epoch')
plt.ylabel('Test Accuracy (%)')
plt.title('Сравнение методов обучения')
plt.legend()
plt.grid(True)
plt.savefig('distillation_comparison.png', dpi=150)
plt.show()
```

Ожидаемые результаты сравнения:

Метод	Параметры	Test Accuracy	Требует Teacher
Baseline Student	∼140K	50–55%	Нет
Student + KD	∼140K	55–65%	Да
DML (Model A)	∼140K	55–65%	Нет
DML (Model B)	∼140K	55–65%	Нет

Таблица 8: Сравнение методов обучения компактных моделей

Домашнее задание

1. Подобрать оптимальную температуру T для дистилляции (попробовать 2, 4, 8, 16).
2. Сравнить SGD и Adam оптимизаторы для Student модели.
3. Эксперимент с коэффициентом α в функции потерь дистилляции.
4. Сохранить веса лучшей модели Student.

Контрольные вопросы

1. Зачем нужно делить логиты на температуру T?

   Ответ: Температура $T > 1$ смягчает распределение вероятностей softmax. Это позволяет Student модели выучить не только правильный класс, но и относительные сходства между классами (dark knowledge), которые скрыты в «хвосте» распределения Teacher.

2. В чём преимущество DML перед классической KD?

   Ответ: DML не требует предобученного Teacher. Две модели обучаются одновременно и взаимообогащают друг друга знаниями. Это полезно, когда нет доступа к большой модели-учителю.

3. Почему Teacher модель должна быть заморожена?

   Ответ: Teacher уже обучен и содержит «знания», которые нужно передать Student. Если разморозить Teacher, он будет изменяться вместе с Student, что нарушает процесс дистилляции (нет стабильного источника знаний).

Занятие 5. Детектирование объектов (YOLO)

Цель занятия

- Изучить архитектуру одностадийных детекторов (YOLO).
- Освоить библиотеку ultralytics в PyTorch.
- Выполнить Fine-tuning модели на custom-датасете.
- Оценить качество через mAP и визуализацию.

Теоретическая часть (1 час)

Принцип работы YOLO

YOLO (You Only Look Once) — одностадийный детектор объектов. В отличие от двухстадийных детекторов (R-CNN), YOLO выполняет классификацию и локализацию за один проход сети.

Основные принципы:

- Изображение делится на сетку $S \times S$ (обычно 13×13)
- Каждая ячейка предсказывает B bounding boxes и классы
- Для каждого бокса предсказываются: координаты (x, y, w, h), уверенность (confidence), классы

Метрики качества

- **IoU (Intersection over Union):** мера перекрытия двух боксов.

$$IoU = \frac{Area(B_{pred} \cap B_{gt})}{Area(B_{pred} \cup B_{gt})}$$

- **Precision:** доля истинных положительных среди всех предсказанных.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall:** доля найденных объектов среди всех существующих.

$$Recall = \frac{TP}{TP + FN}$$

- **mAP (mean Average Precision):** средняя точность по всем классам. mAP@0.5 — при IoU пороге 0.5.

Практическая часть (3 часа)

Шаг 1. Установка библиотеки

```python
# Установка ultralytics (выполнить один раз)
# !pip install ultralytics

import torch
from ultralytics import YOLO

# Проверка GPU
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"Device: {torch.cuda.get_device_name(0)}")
```

Ожидаемый результат:

- Библиотека ultralytics установлена
- GPU доступен для обучения

Шаг 2. Загрузка предобученной модели

```python
# Загрузка предобученной модели YOLOv8n (nano)
# Nano версия: 3.2M параметров, быстрая, подходит для обучения
model = YOLO('yolov8n.pt')

# Информация о модели
print(f"Модель: YOLOv8n")
print(f"Количество параметров: {sum(p.numel() for p in model.model.parameters()):,}")

# Тестовый инференс на изображении
results = model('https://ultralytics.com/images/bus.jpg')

# Отображение результатов
results[0].show()

# Информация о детекциях
print(f"\nНайдено объектов: {len(results[0].boxes)}")
for box in results[0].boxes:
    cls_id = int(box.cls[0])
    cls_name = results[0].names[cls_id]
    conf = float(box.conf[0])
    print(f"  {cls_name}: confidence={conf:.2f}")
```

Ожидаемый результат:

- Загружена модель YOLOv8n (3.2 млн параметров)
- На тестовом изображении обнаружены объекты (люди, автобус)
- Выведены confidence scores для каждого объекта

Шаг 3. Подготовка датасета

Структура датасета для YOLO:

```
dataset/
  images/
    train/
      image1.jpg
      image2.jpg
```

```
    val/
      image3.jpg
  labels/
    train/
      image1.txt  # формат: class x_center y_center width height
      image2.txt
    val/
      image3.txt
  data.yaml
```

Файл data.yaml:

```yaml
# Файл конфигурации датасета
path: ./dataset          # Корневая папка
train: images/train      # Папка с обучающими изображениями
val: images/val          # Папка с валидационными изображениями

nc: 2                    # Количество классов
names: ['ball', 'box']   # Названия классов
```

```python
# Пример создания структуры датасета
import os
import yaml

# Создание директорий
os.makedirs('dataset/images/train', exist_ok=True)
os.makedirs('dataset/images/val', exist_ok=True)
os.makedirs('dataset/labels/train', exist_ok=True)
os.makedirs('dataset/labels/val', exist_ok=True)

# Создание data.yaml
data_config = {
    'path': './dataset',
    'train': 'images/train',
    'val': 'images/val',
    'nc': 2,
    'names': ['ball', 'box']
}

with open('dataset/data.yaml', 'w') as f:
    yaml.dump(data_config, f)

print("Структура датасета создана")
print("Добавьте изображения в dataset/images/train и dataset/images/val")
print("Добавьте разметку в dataset/labels/train и dataset/labels/val")
```

Примечание: Для полноценного обучения необходимо подготовить собственный датасет или использовать готовый (например, с Roboflow).

Шаг 4. Fine-tuning модели

```python
# Обучение на custom датасете
# Параметры обучения:
# - epochs: количество эпох (20 для demo, 100+ для production)
# - imgsz: размер входного изображения (640 стандарт)
# - batch: размер батча (16 для 8GB GPU)
# - device: 0 для GPU

# Если датасет подготовлен:
# results = model.train(
#     data='dataset/data.yaml',
#     epochs=20,
#     imgsz=640,
#     batch=16,
#     device=0
# )
```

```python

# Демонстрация на встроенном датасете COCO (только валидация)
model = YOLO('yolov8n.pt')

# Валидация на COCO (показывает метрики без обучения)
metrics = model.val(data='coco8.yaml')

print("\n=== Метрики на COCO8 ===")
print(f"mAP@0.5: {metrics.box.map50:.4f}")
print(f"mAP@0.5:0.95: {metrics.box.map:.4f}")
print(f"Precision: {metrics.box.mp:.4f}")
print(f"Recall: {metrics.box.mr:.4f}")
```

Ожидаемый результат при обучении:

- Loss уменьшается с каждой эпохой
- mAP@0.5 растёт (целевое значение зависит от датасета)
- Результаты сохраняются в runs/detect/train/

Шаг 5. Оценка и визуализация

```python
# Предсказание на новом изображении
results = model('test_image.jpg', conf=0.25)  # confidence threshold

# Визуализация
for r in results:
    # Сохранение результата
    r.save('prediction.jpg')

    # Вывод детекций
    print(f"Обнаружено объектов: {len(r.boxes)}")
    for box in r.boxes:
        x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
        conf = box.conf[0].cpu().numpy()
        cls = int(box.cls[0].cpu().numpy())
        print(f"  Класс: {r.names[cls]}, Conf: {conf:.2f}, BBox: [{x1:.0f}, {y1:.0f}, {x2:.0f}, {y2:.0f}]")

# Отображение
results[0].show()
```

Шаг 6. Анализ результатов

```python
import matplotlib.pyplot as plt
from PIL import Image

# Визуализация графиков обучения (если было обучение)
# Графики сохраняются в runs/detect/train/

# Пример: отображение результатов
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# 1. Изображение с детекциями
img = Image.open('prediction.jpg')
axes[0].imshow(img)
axes[0].set_title('Результаты детекции')
axes[0].axis('off')

# 2. Пример confusion matrix (если есть)
# cm_path = 'runs/detect/train/confusion_matrix.png'
# if os.path.exists(cm_path):
#     cm = Image.open(cm_path)
#     axes[1].imshow(cm)
# axes[1].set_title('Confusion Matrix')
# axes[1].axis('off')

# 3. Пример PR-кривой
# pr_path = 'runs/detect/train/PR_curve.png'
```

```
# if os.path.exists(pr_path):
#     pr = Image.open(pr_path)
#     axes[2].imshow(pr)
# axes[2].set_title('PR Curve')
# axes[2].axis('off')

plt.tight_layout()
plt.savefig('yolo_results.png', dpi=150)
plt.show()
```

Типичные результаты:

- mAP@0.5: 0.7–0.95 в зависимости от сложности задачи
- Количество детекций зависит от порога confidence
- Fine-tuning улучшает качество на конкретном домене

Домашнее задание

1. Дообучить модель до 50 эпох (если позволяет время Colab).
2. Собрать свой мини-датасет (10–20 фото) и разметить его через Roboflow.
3. Сравнить mAP предобученной модели и дообученной.
4. Попробовать YOLOv8s (small) и сравнить с nano.

Контрольные вопросы

1. В чём разница между One-Stage и Two-Stage детекторами?

   Ответ: One-Stage (YOLO, SSD) выполняет детекцию за один проход сети — быстрее, но менее точно. Two-Stage (Faster R-CNN) сначала генерирует кандидатов (RPN), затем классифицирует — точнее, но медленнее.

2. Что такое IoU и как он считается?

   Ответ: IoU (Intersection over Union) = площадь пересечения / площадь объединения двух боксов. IoU = 1 при полном совпадении, IoU = 0 при отсутствии пересечения.

3. Зачем нужен Non-Maximum Suppression (NMS)?

   Ответ: NMS удаляет дублирующиеся детекции одного объекта. Оставляется бокс с максимальным confidence, остальные с IoU > порога удаляются.

Критерии сдачи занятия

- Код инференса выполняется без ошибок.
- Модель обучается хотя бы 1 эпоху на custom-датасете (если подготовлен).
- Присутствуют графики обучения (loss, mAP).
- Есть визуализация bounding boxes на примерах.

Занятие 6. Трансформеры в компьютерном зрении (ViT)

Цель занятия

- Изучить архитектуру Vision Transformer (ViT).
- Реализовать механизм Patch Embedding и Positional Encoding.
- Обучить ViT на CIFAR-10 и сравнить с CNN.
- Визуализировать карты внимания (Attention Maps).

Теоретическая часть (1 час)

Архитектура ViT

В отличие от CNN, ViT обрабатывает изображение как последовательность патчей (patches):

1. **Patch Embedding:** Разбиение изображения на фиксированные блоки (например, 4×4 пикселя), каждый блок линейно проецируется в вектор.
2. **Positional Encoding:** Добавление информации о позиции патча, так как трансформер не имеет встроённого понятия порядка.
3. **Transformer Encoder:** Слои Multi-Head Attention и MLP.
4. **Classification Head:** MLP на выходе токена [CLS] — специального токена для классификации.

Механизм Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Позволяет модели фокусироваться на важных частях изображения независимо от расстояния между патчами.

Практическая часть (3 часа)

Шаг 1. Подготовка данных (CIFAR-10)

```python
import torch
import torch.nn as nn
from torchvision import datasets, transforms

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ViT требует фиксированного размера входа
# CIFAR-10 (32x32) увеличиваем до 64x64 для лучшего представления патчей
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

train_dataset = datasets.CIFAR10(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = datasets.CIFAR10(
    root='./data', train=False, download=True, transform=transform
)

# Используем подмножество для ускорения
train_subset = torch.utils.data.Subset(train_dataset, range(10000))
test_subset = torch.utils.data.Subset(test_dataset, range(2000))

train_loader = torch.utils.data.DataLoader(
    train_subset, batch_size=64, shuffle=True, num_workers=0
)
test_loader = torch.utils.data.DataLoader(
    test_subset, batch_size=64, shuffle=False, num_workers=0
)

print(f"Изображения: 64x64x3")
print(f"Размер патча 8x8 -> {64//8}x{64//8} = 64 патча")
```

Ожидаемый результат:

- CIFAR-10 изображения масштабированы до 64x64
- При размере патча 8x8 получаем 64 патча + 1 токен [CLS] = 65 токенов

Шаг 2. Реализация Patch Embedding

```python
class PatchEmbedding(nn.Module):
    """
    Преобразование изображения в последовательность патчей.

    Args:
        img_size: размер входного изображения (квадрат)
        patch_size: размер патча (квадрат)
        in_channels: количество каналов входа (3 для RGB)
        embed_dim: размерность эмбеддинга патча

    Принцип работы:
    - Изображение (B, C, H, W) разбивается на патчи
    - Каждый патч проецируется в вектор размерности embed_dim
    - Используем Conv2d с kernel_size=patch_size и stride=patch_size
    """
    def __init__(self, img_size=64, patch_size=8, in_channels=3, embed_dim=128):
        super(PatchEmbedding, self).__init__()
        self.img_size = img_size
        self.patch_size = patch_size
        self.n_patches = (img_size // patch_size) ** 2

        # Conv2d работает как линейная проекция для каждого патча
        self.projection = nn.Conv2d(
            in_channels, embed_dim,
```

```
            kernel_size=patch_size, stride=patch_size
        )

    def forward(self, x):
        # x: (batch, channels, height, width)
        # После projection: (batch, embed_dim, n_patches**0.5, n_patches**0.5)
        x = self.projection(x)

        # Flatten: (batch, embed_dim, n_patches)
        x = x.flatten(2)

        # Transpose: (batch, n_patches, embed_dim)
        x = x.transpose(1, 2)

        return x

# Тест
patch_embed = PatchEmbedding(img_size=64, patch_size=8, embed_dim=128)
dummy_input = torch.randn(1, 3, 64, 64)
output = patch_embed(dummy_input)
print(f"Вход: {dummy_input.shape}")
print(f"Выход: {output.shape}")  # (1, 64, 128) - 64 патча, каждый размерности 128
```

Ожидаемый результат:

- Вход: (1, 3, 64, 64)
- Выход: (1, 64, 128) — 64 патча, каждый представлен вектором размерности 128

Шаг 3. Сборка модели ViT

```
class VisionTransformer(nn.Module):
    """
    Vision Transformer для классификации изображений.

    Компоненты:
    1. Patch Embedding - преобразование изображения в токены
    2. CLS Token - специальный токен для классификации
    3. Positional Encoding - информация о позиции
    4. Transformer Encoder - слои self-attention
    5. Classification Head - полносвязный слой
    """
    def __init__(self, img_size=64, patch_size=8, in_channels=3,
                 num_classes=10, embed_dim=128, depth=4, heads=4):
        super(VisionTransformer, self).__init__()

        # Patch Embedding
        self.patch_embed = PatchEmbedding(img_size, patch_size, in_channels, embed_dim)
        n_patches = self.patch_embed.n_patches

        # CLS Token - обучаемый токен для агрегации информации
        self.cls_token = nn.Parameter(torch.randn(1, 1, embed_dim))

        # Positional Encoding - обучаемые позиции
        # +1 для CLS токена
        self.pos_embed = nn.Parameter(torch.randn(1, n_patches + 1, embed_dim))

        # Transformer Encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim,
            nhead=heads,
            dim_feedforward=embed_dim * 4,  # MLP в 4 раза шире
            dropout=0.1,
            batch_first=True  # Важно: (batch, seq, feature)
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=depth)

        # Classification Head
        self.head = nn.Linear(embed_dim, num_classes)

        # Подсчёт параметров
        self.num_params = sum(p.numel() for p in self.parameters())

```

```python
    def forward(self, x):
        B = x.shape[0]  # batch size

        # Patch Embedding: (B, n_patches, embed_dim)
        x = self.patch_embed(x)

        # Добавляем CLS token
        cls_tokens = self.cls_token.expand(B, -1, -1)  # (B, 1, embed_dim)
        x = torch.cat([cls_tokens, x], dim=1)  # (B, n_patches+1, embed_dim)

        # Добавляем positional encoding
        x = x + self.pos_embed

        # Transformer Encoder
        x = self.transformer(x)

        # Берём только CLS token для классификации
        cls_output = x[:, 0]  # (B, embed_dim)

        # Classification
        return self.head(cls_output)

# Создание модели
model = VisionTransformer(
    img_size=64, patch_size=8, in_channels=3,
    num_classes=10, embed_dim=128, depth=4, heads=4
).to(device)

print(f"ViT параметров: {model.num_params:,}")
print(model)
```

Ожидаемый результат:

- Модель ViT с 400K параметров
- Вход: (B, 3, 64, 64)
- Выход: (B, 10) — логиты для 10 классов

Шаг 4. Обучение модели

```python
from tqdm import tqdm

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0003)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10)

def train_vit(model, train_loader, test_loader, epochs=15):
    history = {'train_loss': [], 'test_acc': []}
    best_acc = 0.0

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for images, labels in tqdm(train_loader, desc=f'Epoch {epoch+1}'):
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        scheduler.step()
        avg_loss = total_loss / len(train_loader)

        # Валидация
        model.eval()
        correct = total = 0
```

```
        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        acc = 100 * correct / total
        history['train_loss'].append(avg_loss)
        history['test_acc'].append(acc)

        best_acc = max(best_acc, acc)

        print(f"Epoch {epoch+1}/{epochs}: Loss={avg_loss:.4f}, Acc={acc:.2f}%")

    return model, history, best_acc

# Обучение
print("=== Обучение ViT ===")
model, history_vit, best_acc_vit = train_vit(model, train_loader, test_loader, epochs=15)
print(f"\nЛучшая точность ViT: {best_acc_vit:.2f}%")
```

Ожидаемый результат:

- ViT сходится медленнее CNN на малых данных
- Accuracy: 45–55% за 15 эпох (на подмножестве CIFAR-10)
- Для лучшего качества ViT требует больше данных и эпох

Шаг 5. Визуализация Attention Maps

```
import matplotlib.pyplot as plt

def visualize_attention(model, image, layer_idx=0):
    """
    Визуализация карт внимания для заданного слоя.

    Показывает, на какие патчи модель обращает внимание.
    """
    model.eval()

    # Регистрируем hook для получения attention weights
    attention_weights = []

    def hook_fn(module, input, output):
        # output[1] содержит attention weights в некоторых реализациях
        # Для упрощения используем выход трансформера
        attention_weights.append(output.detach())

    # Получаем промежуточные представления
    with torch.no_grad():
        x = image.unsqueeze(0).to(device)

        # Patch embedding
        x = model.patch_embed(x)

        # CLS token
        cls_tokens = model.cls_token.expand(1, -1, -1)
        x = torch.cat([cls_tokens, x], dim=1)

        # Positional encoding
        x = x + model.pos_embed

    # Визуализация эмбеддингов
    embeddings = x[0, 1:].cpu().numpy()  # Без CLS token

    # Reshape для отображения (64 патча -> 8x8)
    grid_size = int(embeddings.shape[0] ** 0.5)

    # Сумма активаций по всем измерениям
```

```
    attention_map = embeddings.sum(axis=1).reshape(grid_size, grid_size)

    # Визуализация
    fig, axes = plt.subplots(1, 2, figsize=(10, 4))

    # Исходное изображение
    img = image.cpu().numpy().transpose(1, 2, 0)
    img = (img + 1) / 2  # Денормализация
    axes[0].imshow(img)
    axes[0].set_title('Исходное изображение')
    axes[0].axis('off')

    # Карта внимания
    axes[1].imshow(attention_map, cmap='viridis')
    axes[1].set_title('Attention Map')
    axes[1].axis('off')

    plt.tight_layout()
    plt.savefig('vit_attention.png', dpi=150)
    plt.show()

# Визуализация на примере
sample_image, _ = test_dataset[0]
visualize_attention(model, sample_image)
```

Ожидаемый результат:

- Карта внимания показывает, какие патчи наиболее активны
- Яркие области соответствуют «важным» частям изображения

Домашнее задание

1. Изменить размер патча (4x4 vs 8x8 vs 16x16) и сравнить точность.
2. Добавить аугментацию данных (CutMix или MixUp).
3. Сравнить количество параметров ViT и CNN с аналогичной точностью.
4. Попробовать предобученный ViT из torchvision.models.

Контрольные вопросы

1. Зачем нужен токен [CLS] в ViT?

   Ответ: Токен [CLS] агрегирует информацию со всех патчей через механизм self-attention. Его выходное представление используется для классификации. Аналогично BERT в NLP.

2. Почему ViT требует больше данных для обучения чем CNN?

   Ответ: ViT не имеет встроённых индуктивных смещений (inductive biases) CNN: трансляционной инвариантности и локальности. CNN предполагает, что соседние пиксели связаны. ViT должен выучить эти закономерности из данных.

3. Как вычисляется размерность последовательности патчей?

   Ответ: $n = (H/P) \times (W/P)$, где H, W — размеры изображения, P — размер патча. Для 64x64 и патче 8x8: $(64/8)^2 = 64$ патча.

Критерии сдачи занятия

- Модель ViT обучается без ошибок.
- Accuracy на CIFAR-10 $\geq 45\%$ (за 10–15 эпох на подмножестве).
- Присутствует визуализация внимания.

Занятие 7. Автоэнкодеры (AE) и латентные пространства

Цель занятия

- Изучить архитектуру Автоэнкодера (Encoder-Decoder).
- Научиться визуализировать латентное пространство.
- Реализовать Denoising Autoencoder (DAE).
- Понять применение AE для снижения размерности.

Теоретическая часть (1 час)

Принцип работы AE

Автоэнкодер — нейронная сеть для обучения эффективных представлений данных без учителя (unsupervised learning).

- **Encoder:** Сжимает вход x в латентный вектор z (bottleneck). $z = f_\phi(x)$
- **Decoder:** Восстанавливает $\hat{x}$ из z. $\hat{x} = g_\theta(z)$
- **Loss:** MSE или BCE между x и $\hat{x}$. $L = ||x - \hat{x}||^2$

Размерность латентного пространства меньше размерности входа, что заставляет модель выучивать сжатое представление.

Применение

- Снижение размерности (аналог PCA, но нелинейное)
- Удаление шума (Denoising Autoencoder)
- Детекция аномалий (плохая реконструкция = аномалия)
- Предобучение для других задач

Практическая часть (3 часа)

Шаг 1. Загрузка данных (MNIST)

```python
import torch
import torch.nn as nn
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# MNIST без нормализации (для простоты визуализации)
transform = transforms.ToTensor()

train_dataset = datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = datasets.MNIST(
    root='./data', train=False, download=True, transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=64, shuffle=True, num_workers=0
)
test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=64, shuffle=False, num_workers=0
)

print(f"Train: {len(train_dataset)}, Test: {len(test_dataset)}")
print(f"Размер изображения: 28x28 = 784 пикселя")
```

Ожидаемый результат:

- Загружен MNIST (60000 train, 10000 test)
- Изображения 28x28 = 784 пикселя

Шаг 2. Реализация простого AE

```python
class Autoencoder(nn.Module):
    """
    Простой полносвязный автоэнкодер.

    Архитектура:
    Encoder: 784 -> 256 -> 128 -> latent_dim
    Decoder: latent_dim -> 128 -> 256 -> 784

    latent_dim - размерность латентного пространства
    Чем меньше latent_dim, тем сильнее сжатие, но хуже качество
    """
    def __init__(self, latent_dim=16):
        super(Autoencoder, self).__init__()

        # Encoder: сжимаем 784 -> latent_dim
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim)  # Латентный вектор
        )

        # Decoder: восстанавливаем latent_dim -> 784
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, 28*28),
            nn.Sigmoid()  # Выход в [0, 1]
```

```
        )

    def forward(self, x):
        z = self.encoder(x)       # Кодирование
        x_hat = self.decoder(z)   # Декодирование
        return x_hat, z

    def encode(self, x):
        return self.encoder(x)

    def decode(self, z):
        return self.decoder(z)

# Создание модели
latent_dim = 16  # Сжатие 784 -> 16 (в 49 раз!)
model = Autoencoder(latent_dim=latent_dim).to(device)

# Подсчёт параметров
total_params = sum(p.numel() for p in model.parameters())
print(f"Latent dim: {latent_dim}")
print(f"Параметров модели: {total_params:,}")
print(f"Степень сжатия: {784/latent_dim:.1f}x")
```

Ожидаемый результат:

- Модель с 230K параметров
- Сжатие: 784 → 16 (в 49 раз)

Шаг 3. Обучение AE

```
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

def train_ae(model, train_loader, epochs=10):
    history = {'loss': []}

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for images, _ in train_loader:
            images = images.to(device)

            optimizer.zero_grad()
            x_hat, z = model(images)
            loss = criterion(x_hat, images.view(-1, 28*28))
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        avg_loss = total_loss / len(train_loader)
        history['loss'].append(avg_loss)
        print(f"Epoch {epoch+1}/{epochs}: Loss = {avg_loss:.4f}")

    return history

print("=== Обучение Autoencoder ===")
history = train_ae(model, train_loader, epochs=10)
```

Ожидаемый результат:

- Loss уменьшается от 0.07 до 0.02
- Модель учится реконструировать изображения

Шаг 4. Визуализация реконструкции

```python
def show_reconstruction(model, test_loader, n=10):
    """Визуализация оригиналов и реконструкций"""
    model.eval()

    images, labels = next(iter(test_loader))
    images = images.to(device)

    with torch.no_grad():
        x_hat, z = model(images)

    # Денормализация и reshape
    originals = images.cpu().numpy().squeeze()
    reconstructions = x_hat.cpu().numpy().reshape(-1, 28, 28)

    fig, axes = plt.subplots(2, n, figsize=(n*1.5, 3))

    for i in range(n):
        # Оригинал
        axes[0, i].imshow(originals[i], cmap='gray')
        axes[0, i].axis('off')
        if i == 0:
            axes[0, i].set_title('Оригинал', fontsize=10)

        # Реконструкция
        axes[1, i].imshow(reconstructions[i], cmap='gray')
        axes[1, i].axis('off')
        if i == 0:
            axes[1, i].set_title('Реконструкция', fontsize=10)

    plt.tight_layout()
    plt.savefig('ae_reconstruction.png', dpi=150)
    plt.show()

    print(f"Размер латентного вектора: {z[0].shape}")

show_reconstruction(model, test_loader)
```

Ожидаемый результат:

- Реконструкции похожи на оригиналы, но слегка размытые
- Модель сохраняет основные черты цифр
- Потеря деталей из-за сжатия в латентное пространство

Шаг 5. Визуализация латентного пространства (PCA)

```python
from sklearn.decomposition import PCA
import numpy as np

def visualize_latent_space(model, test_loader):
    """Визуализация латентного пространства с помощью PCA"""
    model.eval()

    all_z = []
    all_labels = []

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            _, z = model(images)
            all_z.append(z.cpu().numpy())
            all_labels.append(labels.numpy())

    all_z = np.concatenate(all_z)
    all_labels = np.concatenate(all_labels)

    # PCA для визуализации (если latent_dim > 2)
    if all_z.shape[1] > 2:
        pca = PCA(n_components=2)
```

```
        z_2d = pca.fit_transform(all_z)
        print(f"PCA объяснённая дисперсия: {pca.explained_variance_ratio_.sum():.2%}")
    else:
        z_2d = all_z

    # Визуализация
    plt.figure(figsize=(10, 8))
    scatter = plt.scatter(z_2d[:, 0], z_2d[:, 1],
                          c=all_labels, cmap='tab10', s=1, alpha=0.5)
    plt.colorbar(scatter, label='Цифра')
    plt.title('Латентное пространство MNIST (PCA)')
    plt.xlabel('Компонента 1')
    plt.ylabel('Компонента 2')
    plt.savefig('ae_latent_space.png', dpi=150)
    plt.show()

visualize_latent_space(model, test_loader)
```

Ожидаемый результат:

- Кластеры, соответствующие разным цифрам
- Похожие цифры (например, 4 и 9) расположены близко
- PCA объясняет значительную часть дисперсии (>70%)

Шаг 6. Denoising Autoencoder (DAE)

```
def train_denoising_ae(model, train_loader, noise_factor=0.5, epochs=10):
    """
    Обучение Denoising Autoencoder.

    На вход подаётся зашумлённое изображение,
    цель - восстановить чистое.
    """
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for images, _ in train_loader:
            images = images.to(device)

            # Добавляем гауссов шум
            noise = torch.randn_like(images) * noise_factor
            noisy_images = images + noise
            noisy_images = torch.clamp(noisy_images, 0, 1)  # Ограничение [0, 1]

            optimizer.zero_grad()
            x_hat, _ = model(noisy_images)
            loss = criterion(x_hat, images.view(-1, 28*28))  # Цель - чистое изображение!
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        avg_loss = total_loss / len(train_loader)
        print(f"DAE Epoch {epoch+1}/{epochs}: Loss = {avg_loss:.4f}")

def show_denoising(model, test_loader, noise_factor=0.5):
    """Демонстрация удаления шума"""
    model.eval()

    images, _ = next(iter(test_loader))
    images = images.to(device)

    # Добавляем шум
    noise = torch.randn_like(images) * noise_factor
    noisy_images = images + noise
    noisy_images = torch.clamp(noisy_images, 0, 1)
```

```python

    with torch.no_grad():
        denoised, _ = model(noisy_images)

    # Визуализация
    n = 8
    fig, axes = plt.subplots(3, n, figsize=(n*1.5, 4.5))

    for i in range(n):
        # Зашумлённое
        axes[0, i].imshow(noisy_images[i].cpu().squeeze(), cmap='gray')
        axes[0, i].axis('off')

        # Очищенное
        axes[1, i].imshow(denoised[i].cpu().reshape(28, 28), cmap='gray')
        axes[1, i].axis('off')

        # Оригинал
        axes[2, i].imshow(images[i].cpu().squeeze(), cmap='gray')
        axes[2, i].axis('off')

    axes[0, 0].set_title('Зашумлённое')
    axes[1, 0].set_title('Очищенное')
    axes[2, 0].set_title('Оригинал')

    plt.tight_layout()
    plt.savefig('dae_denoising.png', dpi=150)
    plt.show()

# Обучение DAE
print("=== Обучение Denoising Autoencoder ===")
dae = Autoencoder(latent_dim=16).to(device)
train_denoising_ae(dae, train_loader, noise_factor=0.5, epochs=10)
show_denoising(dae, test_loader, noise_factor=0.5)
```

Ожидаемый результат:

- DAE успешно удаляет шум
- Реконструкция чище, чем у обычного AE при том же уровне шума
- Модель выучивает робастные признаки, устойчивые к шуму

Домашнее задание

1. Реализовать свёрточный AE (Conv2d + ConvTranspose2d).
2. Сравнить качество реконструкции при latent_dim = 2, 16, 64, 128.
3. Поэкспериментировать с уровнем шума для DAE (0.1, 0.3, 0.5, 0.7).

Контрольные вопросы

1. Зачем нужна функция активации Sigmoid на выходе декодера?

   Ответ: Sigmoid ограничивает выход в диапазон [0, 1], что соответствует диапазону значений пикселей после ToTensor(). Без этого модель могла бы генерировать значения вне этого диапазона.

2. Что произойдёт, если размерность латентного пространства равна входной?

   Ответ: Модель просто научится копировать вход на выход (identity function) без какого-либо сжатия или выучивания полезных признаков. Это лишает AE смысла.

3. Как AE связан с методом главных компонент (PCA)?

   Ответ: Линейный AE (без нелинейных активаций) с MSE loss эквивалентен PCA. AE можно рассматривать как нелинейное обобщение PCA.

Критерии сдачи занятия

- Модель AE обучается и восстанавливает изображения.
- Присутствует график латентного пространства (PCA).
- Реализован эксперимент с шумом (DAE).

Занятие 8. Вариационные автоэнкодеры (VAE)

Цель занятия

- Изучить вероятностную интерпретацию автоэнкодеров.
- Реализовать VAE с репараметризацией (Reparameterization Trick).
- Научиться генерировать новые данные из латентного пространства.
- Освоить интерполяцию между изображениями в latent space.

Теоретическая часть (1 час)

Принцип работы VAE

В отличие от обычного AE, VAE моделирует распределение латентных переменных:

- **Encoder:** Предсказывает параметры распределения — среднее μ и дисперсию σ^2.
- **Reparameterization Trick:** $z = \mu + \sigma \odot \epsilon$, где $\epsilon \sim \mathcal{N}(0, I)$. Это позволяет обратно распространять градиенты через стохастическую операцию сэмплирования.
- **Decoder:** Восстанавливает изображение из сэмплированного z.

Функция потерь VAE

$$L = \underbrace{BCE(x, \hat{x})}_{\text{Reconstruction Loss}} + \beta \cdot \underbrace{KL(\mathcal{N}(\mu, \sigma) || \mathcal{N}(0, 1))}_{\text{Regularization}}$$

KL-дивергенция заставляет латентное распределение быть близким к стандартному нормальному, что обеспечивает непрерывность латентного пространства.

Преимущества VAE

- Непрерывное латентное пространство — можно интерполировать
- Возможность генерации новых сэмплов из $z \sim \mathcal{N}(0, 1)$
- Контролируемая генерация при условных VAE

Практическая часть (3 часа)

Шаг 1. Загрузка данных (FashionMNIST)

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

transform = transforms.ToTensor()

train_dataset = datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = datasets.FashionMNIST(
    root='./data', train=False, download=True, transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=64, shuffle=True, num_workers=0
)
test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=64, shuffle=False, num_workers=0
)

print(f"Train: {len(train_dataset)}, Test: {len(test_dataset)}")
```

Шаг 2. Реализация VAE

```python
class VAE(nn.Module):
    """
    Вариационный автоэнкодер.

    Ключевые отличия от обычного AE:
    1. Encoder выдаёт mu и log_var (параметры распределения)
    2. Используется reparameterization trick для сэмплирования z
    3. KL-дивергенция добавляется к функции потерь
    """
    def __init__(self, latent_dim=20):
        super(VAE, self).__init__()

        # Encoder
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 400),
            nn.ReLU()
        )

        # Параметры латентного распределения
        self.fc_mu = nn.Linear(400, latent_dim)      # Среднее
        self.fc_log_var = nn.Linear(400, latent_dim) # Логарифм дисперсии

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 400),
            nn.ReLU(),
            nn.Linear(400, 28*28),
            nn.Sigmoid()
        )

        self.latent_dim = latent_dim

    def encode(self, x):
        """Кодирование в параметры распределения"""
        h = self.encoder(x)
        mu = self.fc_mu(h)
        log_var = self.fc_log_var(h)
        return mu, log_var

    def reparameterize(self, mu, log_var):
        """
```

```
        Reparameterization trick:
        z = mu + sigma * epsilon, где epsilon ~ N(0, I)

        Это позволяет градиентам проходить через стохастическую операцию.
        """
        std = torch.exp(0.5 * log_var)  # sigma = exp(0.5 * log(sigma^2))
        eps = torch.randn_like(std)      # epsilon ~ N(0, 1)
        z = mu + std * eps               # Сэмплирование
        return z

    def decode(self, z):
        """Декодирование из латентного вектора"""
        return self.decoder(z)

    def forward(self, x):
        """Полный проход: encode -> reparameterize -> decode"""
        mu, log_var = self.encode(x)
        z = self.reparameterize(mu, log_var)
        x_hat = self.decode(z)
        return x_hat, mu, log_var

# Создание модели
latent_dim = 20
vae = VAE(latent_dim=latent_dim).to(device)
print(f"VAE с latent_dim={latent_dim}")
```

Ожидаемый результат:

- Модель VAE с латентным пространством размерности 20
- Encoder выдаёт mu и log_var вместо одного вектора z

Шаг 3. Функция потерь VAE

```
def vae_loss(x, x_hat, mu, log_var, beta=1.0):
    """
    Функция потерь VAE = Reconstruction Loss + KL Divergence.

    Reconstruction Loss: BCE между x и x_hat
    KL Divergence: -0.5 * sum(1 + log_var - mu^2 - exp(log_var))

    Args:
        x: исходное изображение
        x_hat: реконструкция
        mu: среднее латентного распределения
        log_var: логарифм дисперсии
        beta: вес KL-члена (beta-VAE)
    """
    # Reconstruction loss (BCE для изображений [0, 1])
    recon_loss = F.binary_cross_entropy(
        x_hat, x.view(-1, 28*28), reduction='sum'
    )

    # KL divergence: D_KL(N(mu, sigma) || N(0, 1))
    # Аналитическая формула: -0.5 * sum(1 + log_var - mu^2 - exp(log_var))
    kl_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())

    # Общий loss
    loss = recon_loss + beta * kl_loss

    return loss, recon_loss, kl_loss

# Тест функции потерь
x = torch.randn(64, 1, 28, 28).to(device)
x_hat, mu, log_var = vae(x)
loss, recon, kl = vae_loss(x, x_hat, mu, log_var)
print(f"Total Loss: {loss.item():.2f}")
print(f"Recon Loss: {recon.item():.2f}, KL Loss: {kl.item():.2f}")
```

Шаг 4. Обучение модели

```python
from tqdm import tqdm

optimizer = torch.optim.Adam(vae.parameters(), lr=0.001)

def train_vae(model, train_loader, epochs=15, beta=1.0):
    history = {'loss': [], 'recon': [], 'kl': []}

    for epoch in range(epochs):
        model.train()
        total_loss = total_recon = total_kl = 0

        for images, _ in tqdm(train_loader, desc=f'Epoch {epoch+1}'):
            images = images.to(device)

            optimizer.zero_grad()
            x_hat, mu, log_var = model(images)
            loss, recon, kl = vae_loss(images, x_hat, mu, log_var, beta)

            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            total_recon += recon.item()
            total_kl += kl.item()

        n = len(train_loader.dataset)
        history['loss'].append(total_loss / n)
        history['recon'].append(total_recon / n)
        history['kl'].append(total_kl / n)

        print(f"Epoch {epoch+1}: Loss={total_loss/n:.2f}, "
              f"Recon={total_recon/n:.2f}, KL={total_kl/n:.2f}")

    return history

print("=== Обучение VAE ===")
history = train_vae(vae, train_loader, epochs=15, beta=1.0)
```

Ожидаемый результат:

- Reconstruction loss уменьшается
- KL loss стабилизируется (15-25)
- Баланс между реконструкцией и регуляризацией

Шаг 5. Генерация новых изображений

```python
def generate_images(model, n=10):
    """Генерация изображений из случайного латентного вектора"""
    model.eval()

    with torch.no_grad():
        # Сэмплируем из стандартного нормального распределения
        z = torch.randn(n, model.latent_dim).to(device)
        generated = model.decode(z).cpu().numpy()

    # Визуализация
    fig, axes = plt.subplots(1, n, figsize=(n*1.5, 1.5))
    for i in range(n):
        axes[i].imshow(generated[i].reshape(28, 28), cmap='gray')
        axes[i].axis('off')

    plt.suptitle('Сгенерированные изображения (z ~ N(0, 1))')
    plt.tight_layout()
    plt.savefig('vae_generated.png', dpi=150)
    plt.show()

generate_images(vae, n=10)
```

Ожидаемый результат:

- Размытые, но узнаваемые предметы одежды
- Разнообразие генерируемых образов
- Качество ниже, чем у GAN, но латентное пространство более структурировано

Шаг 6. Интерполяция в латентном пространстве

```python
def interpolate(model, test_loader, n_steps=10):
    """
    Интерполяция между двумя изображениями в латентном пространстве.
    """
    model.eval()

    # Берём два изображения
    images, _ = next(iter(test_loader))
    img1, img2 = images[0:1].to(device), images[1:2].to(device)

    with torch.no_grad():
        # Кодируем
        mu1, _ = model.encode(img1)
        mu2, _ = model.encode(img2)

        # Линейная интерполяция
        interpolated = []
        for t in torch.linspace(0, 1, n_steps):
            z = (1 - t) * mu1 + t * mu2
            gen = model.decode(z)
            interpolated.append(gen.cpu().numpy())

    # Визуализация
    fig, axes = plt.subplots(1, n_steps + 2, figsize=((n_steps+2)*1.5, 1.5))

    # Начальное изображение
    axes[0].imshow(img1.cpu().squeeze(), cmap='gray')
    axes[0].set_title('Начало')
    axes[0].axis('off')

    # Интерполяция
    for i, img in enumerate(interpolated):
        axes[i+1].imshow(img.reshape(28, 28), cmap='gray')
        axes[i+1].axis('off')

    # Конечное изображение
    axes[-1].imshow(img2.cpu().squeeze(), cmap='gray')
    axes[-1].set_title('Конец')
    axes[-1].axis('off')

    plt.suptitle('Интерполяция в латентном пространстве')
    plt.tight_layout()
    plt.savefig('vae_interpolation.png', dpi=150)
    plt.show()

interpolate(vae, test_loader, n_steps=8)
```

Ожидаемый результат:

- Плавный переход между изображениями
- Все промежуточные образы реалистичны
- Демонстрация непрерывности латентного пространства

Шаг 7. Визуализация функции потерь

```
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(history['loss'], label='Total Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Общая функция потерь')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history['recon'], label='Reconstruction')
plt.plot(history['kl'], label='KL Divergence')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Компоненты функции потерь')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig('vae_loss.png', dpi=150)
plt.show()
```

Домашнее задание

1. Эксперимент с β-VAE: попробовать $\beta = 0.1, 1.0, 5.0, 10.0$.
2. Реализовать Conditional VAE (генерация по классу).
3. Сравнить качество генерации VAE и обычного AE.

Контрольные вопросы

1. Зачем нужен Reparameterization Trick?

   Ответ: Нельзя обратно распространять градиенты через случайную операцию сэмплирования. Reparameterization Trick выделяет стохастическую часть (ϵ) отдельно, позволяя градиентам проходить через детерминированные μ и σ.

2. Почему KL-дивергенция важна для генерации?

   Ответ: KL-дивергенция заставляет латентное распределение быть близким к стандартному нормальному. Это гарантирует, что при генерации из $z \sim \mathcal{N}(0, 1)$ мы получим осмысленные изображения.

3. В чём разница между VAE и обычным AE?

   Ответ: AE детерминированно кодирует вход в точку латентного пространства. VAE моделирует распределение, что позволяет генерировать новые данные и интерполировать между образами. Латентное пространство VAE непрерывно и структурировано.

Критерии сдачи занятия

- Модель VAE обучается без ошибок.
- Присутствует визуализация сгенерированных изображений.

- Реализована интерполяция в латентном пространстве.

Занятие 9. Генеративно-состязательные сети (GAN)

Цель занятия

- Изучить архитектуру GAN (Generator + Discriminator).
- Реализовать DCGAN на FashionMNIST.
- Понять проблемы обучения GAN и методы их решения.
- Сравнить GAN с VAE по качеству генерации.

Теоретическая часть (1 час)

Принцип работы GAN

GAN состоит из двух нейронных сетей:

- **Generator (G):** Создаёт изображения из случайного шума z. Цель: обмануть дискриминатор.
- **Discriminator (D):** Классифицирует изображения как настоящие или сгенерированные. Цель: правильно различать настоящие и фейковые.

Минимаксная игра:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

Проблемы обучения GAN

- **Mode Collapse:** Генератор производит ограниченное разнообразие образов.
- **Нестабильность:** Дискриминатор может «победить» слишком быстро.
- **Vanishing Gradients:** Когда D слишком хорош, градиенты для G исчезают.

DCGAN Architecture

Deep Convolutional GAN — стандартная архитектура с использованием:

- ConvTranspose2d (upside-down convolution) в генераторе
- Strided Conv2d вместо пулинга
- BatchNorm (кроме выходных слоёв)
- ReLU/LeakyReLU активации

Практическая часть (3 часа)

Шаг 1. Загрузка данных (FashionMNIST)

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
from torchvision.utils import make_grid
import matplotlib.pyplot as plt

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # Нормализация к [-1, 1]
])

train_dataset = datasets.FashionMNIST(
    root='./data', train=True, download=True, transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=64, shuffle=True, num_workers=0
)

print(f"Train samples: {len(train_dataset)}")
print(f"Размер изображения: 28x28, нормализация: [-1, 1]")
```

Шаг 2. Реализация Генератора

```python
class Generator(nn.Module):
    """
    Генератор DCGAN для FashionMNIST.

    Архитектура: latent_dim -> ConvTranspose -> 28x28

    Вход: (batch, latent_dim) - случайный вектор
    Выход: (batch, 1, 28, 28) - сгенерированное изображение
    """
    def __init__(self, latent_dim=100):
        super(Generator, self).__init__()

        # Полносвязный слой + reshape
        self.fc = nn.Sequential(
            nn.Linear(latent_dim, 128 * 7 * 7),
            nn.ReLU()
        )

        # Свёрточные слои (upsampling)
        self.conv = nn.Sequential(
            # 7x7 -> 14x14
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            # 14x14 -> 28x28
            nn.ConvTranspose2d(64, 1, kernel_size=4, stride=2, padding=3),
            nn.Tanh()  # Выход в [-1, 1]
        )

        self.latent_dim = latent_dim

    def forward(self, z):
        # z: (batch, latent_dim)
        x = self.fc(z)
        x = x.view(-1, 128, 7, 7)  # Reshape в (batch, 128, 7, 7)
        x = self.conv(x)            # (batch, 1, 28, 28)
        return x

# Тест генератора
G = Generator(latent_dim=100).to(device)
z = torch.randn(1, 100).to(device)
```

```
fake_img = G(z)
print(f"Вход: {z.shape}")
print(f"Выход: {fake_img.shape}")  # (1, 1, 28, 28)
```

Ожидаемый результат:

- Вход: (batch, 100) — латентный вектор
- Выход: (batch, 1, 28, 28) — сгенерированное изображение

Шаг 3. Реализация Дискриминатора

```
class Discriminator(nn.Module):
    """
    Дискриминатор DCGAN.

    Классифицирует изображения как настоящие (1) или сгенерированные (0).

    Вход: (batch, 1, 28, 28) - изображение
    Выход: (batch, 1) - вероятность "настоящего"
    """
    def __init__(self):
        super(Discriminator, self).__init__()

        self.conv = nn.Sequential(
            # 28x28 -> 14x14
            nn.Conv2d(1, 64, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            # 14x14 -> 7x7
            nn.Conv2d(64, 128, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True)
        )

        # Классификатор
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 7 * 7, 1),
            nn.Sigmoid()  # Вероятность [0, 1]
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.fc(x)
        return x

# Тест дискриминатора
D = Discriminator().to(device)
output = D(fake_img)
print(f"Вход: {fake_img.shape}")
print(f"Выход: {output.shape}")  # (1, 1)
```

Шаг 4. Инициализация весов

```
def weights_init(m):
    """
    Инициализация весов для DCGAN.

    Согласно статье DCGAN:
    - Conv/ConvTranspose: N(0, 0.02)
    - BatchNorm: mean=1, std=0.02
    """
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

```

```
# Применяем инициализацию
G = Generator(latent_dim=100).to(device)
D = Discriminator().to(device)
G.apply(weights_init)
D.apply(weights_init)

print("Веса инициализированы")
```

Шаг 5. Функции потерь и оптимизаторы

```
# Функция потерь
criterion = nn.BCELoss()

# Оптимизаторы (раздельные для G и D)
lr = 0.0002
beta1 = 0.5  # Momentum для Adam
optimizer_G = torch.optim.Adam(G.parameters(), lr=lr, betas=(beta1, 0.999))
optimizer_D = torch.optim.Adam(D.parameters(), lr=lr, betas=(beta1, 0.999))

latent_dim = 100
print(f"Learning rate: {lr}")
print(f"Beta1: {beta1}")
```

Шаг 6. Цикл обучения GAN

```
from tqdm import tqdm

def train_gan(G, D, train_loader, epochs=30):
    history = {'G_loss': [], 'D_loss': []}

    for epoch in range(epochs):
        G.train()
        D.train()

        g_loss_epoch = 0
        d_loss_epoch = 0

        for real_images, _ in tqdm(train_loader, desc=f'Epoch {epoch+1}'):
            batch_size = real_images.size(0)
            real_images = real_images.to(device)

            # ===== Обучение Дискриминатора =====
            # Цель: D(real) -> 1, D(fake) -> 0

            optimizer_D.zero_grad()

            # Real images
            real_labels = torch.ones(batch_size, 1).to(device)
            d_real = D(real_images)
            loss_d_real = criterion(d_real, real_labels)

            # Fake images
            z = torch.randn(batch_size, latent_dim).to(device)
            fake_images = G(z).detach()  # detach() - не считаем градиенты для G
            fake_labels = torch.zeros(batch_size, 1).to(device)
            d_fake = D(fake_images)
            loss_d_fake = criterion(d_fake, fake_labels)

            # Общий loss D
            loss_d = loss_d_real + loss_d_fake
            loss_d.backward()
            optimizer_D.step()

            # ===== Обучение Генератора =====
            # Цель: D(fake) -> 1 (обмануть D)

            optimizer_G.zero_grad()

            z = torch.randn(batch_size, latent_dim).to(device)
            fake_images = G(z)

            # G хочет, чтобы D думал, что fakes - это реальные
```

```
            g_labels = torch.ones(batch_size, 1).to(device)
            d_fake_for_g = D(fake_images)
            loss_g = criterion(d_fake_for_g, g_labels)

            loss_g.backward()
            optimizer_G.step()

            g_loss_epoch += loss_g.item()
            d_loss_epoch += loss_d.item()

        # Сохраняем средние потери
        history['G_loss'].append(g_loss_epoch / len(train_loader))
        history['D_loss'].append(d_loss_epoch / len(train_loader))

        print(f"Epoch {epoch+1}: G_loss={history['G_loss'][-1]:.4f}, "
              f"D_loss={history['D_loss'][-1]:.4f}")

        # Генерация примеров каждые 5 эпох
        if (epoch + 1) % 5 == 0:
            generate_samples(G, epoch + 1)

    return history

def generate_samples(G, epoch, n=16):
    """Генерация и визуализация примеров"""
    G.eval()
    with torch.no_grad():
        z = torch.randn(n, latent_dim).to(device)
        fake_images = G(z).cpu()

    # Денормализация [-1, 1] -> [0, 1]
    fake_images = fake_images * 0.5 + 0.5

    grid = make_grid(fake_images, nrow=4, padding=2)
    plt.figure(figsize=(6, 6))
    plt.imshow(grid.permute(1, 2, 0).squeeze(), cmap='gray')
    plt.title(f'Generated Images - Epoch {epoch}')
    plt.axis('off')
    plt.savefig(f'gan_epoch_{epoch}.png', dpi=150)
    plt.show()

# Обучение
print("=== Обучение GAN ===")
history = train_gan(G, D, train_loader, epochs=30)
```

Ожидаемый результат:

- D_loss сначала низкий, затем стабилизируется
- G_loss может расти (это нормально — D становится лучше)
- Качество генераций улучшается от шума к узнаваемым предметам

Шаг 7. Визуализация прогресса генерации

```
# Финальная генерация
G.eval()
n = 25
with torch.no_grad():
    z = torch.randn(n, latent_dim).to(device)
    generated = G(z).cpu() * 0.5 + 0.5

fig, axes = plt.subplots(5, 5, figsize=(8, 8))
for i, ax in enumerate(axes.flat):
    ax.imshow(generated[i].squeeze(), cmap='gray')
    ax.axis('off')

plt.suptitle('Сгенерированные изображения FashionMNIST')
plt.tight_layout()
plt.savefig('gan_final.png', dpi=150)
plt.show()
```

Шаг 8. Графики функции потерь

```
plt.figure(figsize=(10, 4))

plt.plot(history['G_loss'], label='Generator Loss')
plt.plot(history['D_loss'], label='Discriminator Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Кривые обучения GAN')
plt.legend()
plt.grid(True)
plt.savefig('gan_loss.png', dpi=150)
plt.show()
```

Шаг 9. Сравнение VAE и GAN

```
print("=== Сравнение VAE и GAN ===\n")

comparison = """
| Характеристика         | VAE                    | GAN                    |
|------------------------|------------------------|------------------------|
| Качество генерации     | Размытые изображения   | Чёткие изображения     |
| Латентное пространство | Непрерывное            | Может быть разрывным   |
| Интерполяция           | Плавная                | Может давать артефакты |
| Обучение               | Стабильное             | Нестабильное           |
| Дивергенция KL         | Есть (регуляризация)   | Нет                    |
| Mode Collapse          | Нет                    | Возможна               |
| Генерация по классу    | Conditional VAE        | Conditional GAN        |
"""

print(comparison)
```

Домашнее задание

1. Добавить Label Smoothing для стабилизации обучения.
2. Попробовать WGAN (Wasserstein GAN) для улучшения стабильности.
3. Реализовать Conditional GAN для генерации по классу.

Контрольные вопросы

1. Почему GAN сложнее обучать, чем VAE?

   Ответ: GAN — это минимаксная игра, где две сети конкурируют. Равновесие достигается не всегда. Дискриминатор может стать слишком сильным, градиенты для генератора исчезнут. VAE оптимизирует одну функцию потерь, что более стабильно.

2. Что такое Mode Collapse?

   Ответ: Mode Collapse — когда генератор производит ограниченное разнообразие образов (например, только один тип одежды). Причина: генератор нашёл «лазейку», которая обманывает дискриминатор, и exploit'ит её.

3. Почему используем LeakyReLU в дискриминаторе?

   Ответ: LeakyReLU пропускает небольшой градиент для отрицательных значений: $f(x) = \max(\alpha x, x)$. Это предотвращает «умирание» нейронов, когда градиенты становятся нулевыми.

Критерии сдачи занятия

- GAN обучается без краха (mode collapse).
- Присутствуют графики G_loss и D_loss.
- Сгенерированные изображения узнаваемы.

Занятие 10. Диффузионные модели (DDPM / LDM)

Цель занятия

- Изучить принцип работы диффузионных моделей.
- Реализовать простой DDPM на MNIST.
- Понять процесс прямой и обратной диффузии.
- Сравнить с другими генеративными моделями.

Теоретическая часть (1 час)

Принцип диффузии

Диффузионные модели работают в два этапа:

1. **Forward Process (добавление шума):** Постепенно добавляем гауссов шум к изображению за T шагов, пока оно не станет чистым шумом.

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

2. **Reverse Process (удаление шума):** Нейронная сеть учится предсказывать шум на каждом шаге и постепенно удаляет его.

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

Latent Diffusion Models (LDM)

LDM (Stable Diffusion) выполняет диффузию не в пространстве пикселей, а в латентном пространстве автоэнкодера:

- VAE кодирует изображение в компактный латентный код
- Диффузия происходит в латентном пространстве (быстрее)
- VAE декодирует результат обратно в изображение

Практическая часть (3 часа)

Шаг 1. Подготовка данных и гиперпараметров

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import math

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Гиперпараметры диффузии
T = 300  # Количество шагов диффузии
img_size = 28
channels = 1

# Загрузка MNIST
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))  # [-1, 1]
])

train_dataset = datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=64, shuffle=True, num_workers=0
)

print(f"Шагов диффузии: {T}")
print(f"Размер изображения: {img_size}x{img_size}")
```

Шаг 2. Функции добавления шума

```python
def linear_beta_schedule(timesteps, start=0.0001, end=0.02):
    """Линейное расписание beta (управляет количеством шума)"""
    return torch.linspace(start, end, timesteps)

# Параметры диффузии
betas = linear_beta_schedule(T).to(device)

# Альфы и их накопленные произведения
alphas = 1.0 - betas
alphas_cumprod = torch.cumprod(alphas, dim=0)
alphas_cumprod_prev = F.pad(alphas_cumprod[:-1], (1, 0), value=1.0)

# Коэффициенты для сэмплирования
sqrt_recip_alphas = torch.sqrt(1.0 / alphas)
sqrt_alphas_cumprod = torch.sqrt(alphas_cumprod)
sqrt_one_minus_alphas_cumprod = torch.sqrt(1.0 - alphas_cumprod)
posterior_variance = betas * (1.0 - alphas_cumprod_prev) / (1.0 - alphas_cumprod)

def extract(a, t, x_shape):
    """Извлечение коэффициентов для временного шага t"""
    batch_size = t.shape[0]
    out = a.gather(-1, t.cpu())
    return out.reshape(batch_size, *((1,) * (len(x_shape) - 1))).to(device)

def q_sample(x_0, t, noise=None):
    """
    Forward process: добавление шума к x_0 на шаге t.

    Формула: x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * noise

    Это позволяет за один шаг получить зашумлённое изображение
    на любом временном шаге t.
    """
    if noise is None:
        noise = torch.randn_like(x_0)

```

```
    sqrt_alpha_t = extract(sqrt_alphas_cumprod, t, x_0.shape)
    sqrt_one_minus_alpha_t = extract(sqrt_one_minus_alphas_cumprod, t, x_0.shape)

    return sqrt_alpha_t * x_0 + sqrt_one_minus_alpha_t * noise

# Тест: визуализация зашумления
def show_noise_process(x_0, steps=[0, 50, 100, 200, 299]):
    """Визуализация процесса добавления шума"""
    fig, axes = plt.subplots(1, len(steps), figsize=(len(steps)*2, 2))

    for i, t in enumerate(steps):
        t_tensor = torch.tensor([t]).to(device)
        x_t = q_sample(x_0, t_tensor)

        img = x_t.cpu().squeeze() * 0.5 + 0.5
        axes[i].imshow(img, cmap='gray')
        axes[i].set_title(f't={t}')
        axes[i].axis('off')

    plt.suptitle('Forward Diffusion Process')
    plt.tight_layout()
    plt.savefig('diffusion_forward.png', dpi=150)
    plt.show()

# Тест на одном изображении
x_0, _ = next(iter(train_loader))
x_0 = x_0[0:1].to(device)
show_noise_process(x_0)
```

Ожидаемый результат:

- t=0: чёткое изображение
- t=100: размытое, но узнаваемое
- t=200: почти полностью шум
- t=299: чистый гауссов шум

Шаг 3. Модель UNet (Упрощённая)

```
class SinusoidalPositionEmbeddings(nn.Module):
    """Sinusoidal позиционное кодирование для временного шага t"""
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, t):
        device = t.device
        half_dim = self.dim // 2
        embeddings = math.log(10000) / (half_dim - 1)
        embeddings = torch.exp(torch.arange(half_dim, device=device) * -embeddings)
        embeddings = t[:, None] * embeddings[None, :]
        embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
        return embeddings

class SimpleUNet(nn.Module):
    """
    Упрощённая UNet для предсказания шума.

    Структура:
    - Encoder: уменьшение размерности
    - Bottleneck: обработка в латентном пространстве
    - Decoder: восстановление размерности
    - Time embedding: информация о текущем шаге t
    """
    def __init__(self, channels=1, time_dim=256):
        super().__init__()

        # Time embedding
        self.time_mlp = nn.Sequential(
            SinusoidalPositionEmbeddings(time_dim),
```

```
            nn.Linear(time_dim, time_dim),
            nn.ReLU()
        )

        # Encoder
        self.conv1 = nn.Conv2d(channels, 64, 3, padding=1)
        self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
        self.conv3 = nn.Conv2d(128, 256, 3, padding=1)
        self.pool = nn.MaxPool2d(2)

        # Bottleneck
        self.bottleneck = nn.Conv2d(256, 256, 3, padding=1)

        # Decoder
        self.upconv1 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.upconv2 = nn.ConvTranspose2d(256, 64, 2, stride=2)
        self.conv_out = nn.Conv2d(128, channels, 3, padding=1)

        # Time projection layers
        self.time_proj1 = nn.Linear(time_dim, 64)
        self.time_proj2 = nn.Linear(time_dim, 128)
        self.time_proj3 = nn.Linear(time_dim, 256)

    def forward(self, x, t):
        # Time embedding
        t_emb = self.time_mlp(t)

        # Encoder с time conditioning
        x1 = F.relu(self.conv1(x) + self.time_proj1(t_emb)[:, :, None, None])
        x2 = self.pool(x1)

        x2 = F.relu(self.conv2(x2) + self.time_proj2(t_emb)[:, :, None, None])
        x3 = self.pool(x2)

        x3 = F.relu(self.conv3(x3) + self.time_proj3(t_emb)[:, :, None, None])

        # Bottleneck
        x3 = F.relu(self.bottleneck(x3))

        # Decoder
        x = self.upconv1(x3)
        x = torch.cat([x, x2], dim=1)  # Skip connection
        x = F.relu(x)

        x = self.upconv2(x)
        x = torch.cat([x, x1], dim=1)  # Skip connection
        x = F.relu(x)

        return self.conv_out(x)

# Создание модели
model = SimpleUNet(channels=1).to(device)
print(f"Параметров модели: {sum(p.numel() for p in model.parameters()):,}")
```

Шаг 4. Цикл обучения

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

def p_losses(denoise_model, x_0, t):
    """
    Функция потерь для обучения модели денойзинга.

    Модель учится предсказывать шум epsilon,
    который был добавлен к x_0 для получения x_t.
    """
    noise = torch.randn_like(x_0)

    # Зашумляем изображение
    x_t = q_sample(x_0, t, noise)

    # Модель предсказывает шум
    predicted_noise = denoise_model(x_t, t)

    # Loss = MSE между истинным и предсказанным шумом
    loss = F.mse_loss(noise, predicted_noise)
```

```python

    return loss

def train_diffusion(model, train_loader, epochs=10):
    history = {'loss': []}

    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for batch, _ in train_loader:
            batch = batch.to(device)
            t = torch.randint(0, T, (batch.size(0),), device=device).long()

            optimizer.zero_grad()
            loss = p_losses(model, batch, t)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

        avg_loss = total_loss / len(train_loader)
        history['loss'].append(avg_loss)
        print(f"Epoch {epoch+1}/{epochs}: Loss = {avg_loss:.4f}")

    return history

print("=== Обучение Diffusion Model ===")
history = train_diffusion(model, train_loader, epochs=10)
```

Ожидаемый результат:

- Loss уменьшается от 0.1 до 0.02
- Модель учится предсказывать шум на разных временных шагах

Шаг 5. Процесс сэмплирования (Генерация)

```python
@torch.no_grad()
def p_sample(model, x, t, t_index):
    """
    Один шаг обратного процесса денойзинга.

    Формула: x_{t-1} = 1/sqrt(alpha_t) * (x_t - beta_t/sqrt(1-alpha_bar_t) * epsilon_theta) + noise
    """
    betas_t = extract(betas, t, x.shape)
    sqrt_one_minus_alphas_cumprod_t = extract(sqrt_one_minus_alphas_cumprod, t, x.shape)
    sqrt_recip_alphas_t = extract(sqrt_recip_alphas, t, x.shape)

    # Предсказание шума моделью
    model_mean = sqrt_recip_alphas_t * (
        x - betas_t * model(x, t) / sqrt_one_minus_alphas_cumprod_t
    )

    if t_index == 0:
        return model_mean
    else:
        posterior_variance_t = extract(posterior_variance, t, x.shape)
        noise = torch.randn_like(x)
        return model_mean + torch.sqrt(posterior_variance_t) * noise

@torch.no_grad()
def sample(model, image_size=28, batch_size=16, channels=1):
    """
    Полный процесс генерации изображений.

    Начинаем с чистого шума и постепенно удаляем его.
    """
    model.eval()

    # Начинаем с чистого шума
    x = torch.randn(batch_size, channels, image_size, image_size).to(device)

```

```
    # Итеративно удаляем шум
    for t in reversed(range(T)):
        t_batch = torch.full((batch_size,), t, device=device, dtype=torch.long)
        x = p_sample(model, x, t_batch, t)

    return x

# Генерация изображений
print("Генерация изображений...")
generated = sample(model, batch_size=16)

# Визуализация
generated = generated.cpu() * 0.5 + 0.5  # Денормализация
fig, axes = plt.subplots(4, 4, figsize=(6, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(generated[i].squeeze(), cmap='gray')
    ax.axis('off')

plt.suptitle('Сгенерированные изображения (DDPM)')
plt.tight_layout()
plt.savefig('diffusion_generated.png', dpi=150)
plt.show()
```

Ожидаемый результат:

- Размытые, но узнаваемые цифры
- Качество ниже, чем у GAN, но обучение более стабильное
- Для лучшего качества нужно больше эпох и большая модель

Шаг 6. Сравнение генеративных моделей

```
print("=== Сравнение генеративных моделей ===\n")

comparison = """
| Модель    | Качество      | Разнообразие  | Стабильность  | Скорость      |
|-----------|---------------|---------------|---------------|---------------|
| VAE       | Низкое        | Высокое       | Высокая       | Быстрая       |
| GAN       | Высокое       | Среднее*      | Низкая        | Быстрая       |
| Diffusion | Очень высокое | Высокое       | Высокая       | Медленная     |

* GAN страдает от Mode Collapse
"""

print(comparison)

print("\nВыводы:")
print("1. VAE - простая и стабильная, но размытые изображения")
print("2. GAN - острые изображения, но нестабильное обучение")
print("3. Diffusion - лучшее качество, но медленная генерация")
print("4. Для production: Diffusion (Stable Diffusion) или GAN (StyleGAN)")
```

Домашнее задание

1. Увеличить количество эпох до 50 и сравнить качество генерации.
2. Попробовать другое расписание beta (cosine, quadratic).
3. Добавить attention слои в UNet для улучшения качества.
4. Реализовать conditional generation (по классу цифры).

Контрольные вопросы

1. Почему диффузионные модели обучаются стабильнее, чем GAN?

   Ответ: Диффузия оптимизирует одну функцию потерь (MSE между истинным и предсказанным шумом). GAN — это минимаксная игра двух сетей, которая может не достигать равновесия.

2. Зачем нужен time embedding в UNet?

   Ответ: Модель должна знать, на каком шаге диффузии она находится. Количество шума и стратегия денойзинга зависят от t. Time embedding позволяет модели условно генерировать в зависимости от времени.

3. Почему генерация в diffusion медленная?

   Ответ: Для генерации одного изображения нужно выполнить T шагов денойзинга (обычно 1000). Каждый шаг — это проход через UNet. GAN генерирует за один проход.

Критерии сдачи занятия

- Модель обучается без ошибок.
- Присутствуют визуализации forward process.
- Сгенерированные изображения содержат узнаваемые паттерны.

Мини-проект: Методические указания

Общие сведения

Мини-проект является итоговой работой по курсу «Нейронные сети». Студент самостоятельно выбирает тему из предложенного списка или предлагает свою, согласовав с преподавателем.

Срок выполнения: после завершения 50 часов практикума.
Формат: Colab-ноутбук + презентация (5–7 слайдов).
Защита: устная презентация + демонстрация модели.

Этапы выполнения

1. **Выбор темы** (неделя 1): согласование с преподавателем.
2. **Сбор данных** (неделя 2): подготовка датасета, EDA.
3. **Реализация** (недели 3–4): разработка модели, обучение.
4. **Анализ результатов** (неделя 5): визуализация, выводы.
5. **Оформление отчёта** (неделя 6): написание отчёта, подготовка презентации.
6. **Защита** (неделя 7): презентация результатов.

Требования к коду

- Код должен быть воспроизводимым (фиксированный seed).
- Все зависимости указаны в начале ноутбука.
- Комментарии к ключевым этапам.
- Сохранение весов модели (.pth файлы).
- Визуализация процесса обучения.

Требования к отчёту

1. Титульный лист.
2. Аннотация (цель, задачи, результаты).
3. Введение (постановка задачи).

4. Методы (описание архитектуры и подхода).
5. Эксперименты (датасет, метрики, результаты).
6. Визуализация (графики, изображения).
7. Выводы.
8. Список литературы.

Материалы курса в GitHub

Репозиторий курса

Все материалы курса (шаблоны ноутбуков, примеры решений, обновления) публикуются в открытом репозитории GitHub:

https://github.com/V171/neural-networks-practicum

Структура репозитория

```
neural-networks-practicum/
|-- README.md                    # Описание курса
|-- notebooks/                   # Шаблоны ноутбуков
|   |-- 01_cnn_introduction.ipynb
|   |-- 02_overfitting_regularization.ipynb
|   |-- 03_transfer_learning.ipynb
|   |-- 04_distillation_mutual_learning.ipynb
|   |-- 05_yolo_detection.ipynb
|   |-- 06_vit_transformer.ipynb
|   |-- 07_autoencoder.ipynb
|   |-- 08_vae.ipynb
|   |-- 09_gan.ipynb
|   |-- 10_diffusion.ipynb
|-- solutions/                   # Примеры решений (после дедлайнов)
|-- data/                        # Ссылки на датасеты
|-- utils/                       # Вспомогательные функции
|-- docs/                        # Дополнительная документация
|   |-- practicum.pdf            # PDF версия практикума
```

Как работать с репозиторием

Клонирование

```
git clone https://github.com/V171/neural-networks-practicum.git
cd neural-networks-practicum
```

Получение обновлений

```
git pull origin main
```

Отправка решений (форк)

```
# Форк репозитория на GitHub
# Клонирование своего форка
git clone https://github.com/YOUR_USERNAME/neural-networks-practicum.git

# Создание ветки для работы
git checkout -b session-01

# Добавление изменений
git add notebooks/01_cnn_introduction.ipynb
git commit -m "Completed session 1"

# Отправка в свой форк
git push origin session-01

# Создание Pull Request на GitHub
```

Приложение А. Шаблон отчёта по лабораторной работе

Структура отчёта

1. **Название работы:** Занятие №X. Название темы.
2. **Цель работы:** Краткое описание целей занятия.
3. **Теоретическая часть:** Основные понятия и формулы.
4. **Ход работы:**
   - Описание каждого шага.
   - Код с комментариями.
   - Результаты выполнения.
5. **Результаты:**
   - Графики обучения (loss, accuracy).
   - Визуализации (предсказания, карты признаков).
   - Итоговые метрики.
6. **Выводы:** Анализ результатов, ответы на контрольные вопросы.
7. **Список литературы:** Использованные источники.

Приложение Б. Чек-лист перед сдачей

Проверка ноутбука

- ☐ Ноутбук запускается без ошибок (Runtime → Run all).
- ☐ Все ячейки выполнены последовательно.
- ☐ Присутствуют все обязательные графики.
- ☐ Метрики достигают пороговых значений.
- ☐ Есть текстовые выводы в Markdown-ячейках.
- ☐ Комментарии в коде на русском или английском.
- ☐ Фиксированный random seed для воспроизводимости.
- ☐ Ноутбук сохранён в формате .ipynb.

Проверка отчёта

- ☐ Титульный лист оформлен по стандарту.
- ☐ Все разделы присутствуют.
- ☐ Графики подписаны и имеют легенду.
- ☐ Выводы содержательны.
- ☐ Список литературы оформлен по ГОСТ.

Приложение В. Примеры тем мини-проектов

Тема 1: Сравнение генеративных моделей

Задача: Сравнить AE, VAE, GAN и Diffusion на одном датасете (например, CelebA или CIFAR-10).

Метрики: FID (Fréchet Inception Distance), IS (Inception Score), визуальное качество.

Ожидаемые результаты: Таблица сравнения, анализ преимуществ и недостатков каждой модели.

Тема 2: Детектирование специфических объектов

Задача: Обучить YOLO на custom-датасете (сбор и разметка данных самостоятельно).

Метрики: mAP@0.5, mAP@0.5:0.95, скорость инференса.

Ожидаемые результаты: Working demo, анализ ошибок детекции.

Тема 3: Transfer Learning для медицинской диагностики

Задача: Классификация медицинских изображений (рентген, МРТ) с использованием Transfer Learning.

Метрики: Accuracy, Sensitivity, Specificity, AUC-ROC.

Ожидаемые результаты: Сравнение стратегий (Feature Extraction vs Fine-tuning), анализ важности аугментации.

Тема 4: Интерполяция в латентном пространстве VAE

Задача: Исследовать структуру латентного пространства VAE для генерации промежуточных образов.

Метрики: Качество интерполяции (визуальная оценка), плавность переходов.

Ожидаемые результаты: Визуализации интерполяций, анализ семантических направлений в латентном пространстве.

Заключение

Данный практикум охватывает ключевые темы современной глубокой обработки изображений: от базовых свёрточных сетей до современных генеративных моделей. Каждое занятие построено по принципу «от теории к практике» и позволяет студентам получить навыки работы с фреймворком PyTorch.

По завершении курса студенты будут способны:

- Разрабатывать и обучать нейронные сети для задач классификации и детекции
- Применять Transfer Learning для работы с малыми данными
- Реализовывать генеративные модели (VAE, GAN, Diffusion)
- Анализировать и интерпретировать результаты обучения
- Самостоятельно исследовать новые архитектуры и методы

Материалы курса постоянно обновляются в репозитории GitHub. Вопросы и предложения можно направлять преподавателям курса.

Успехов в изучении нейронных сетей!